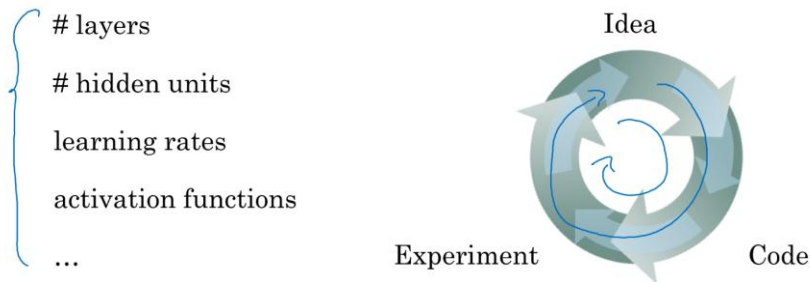


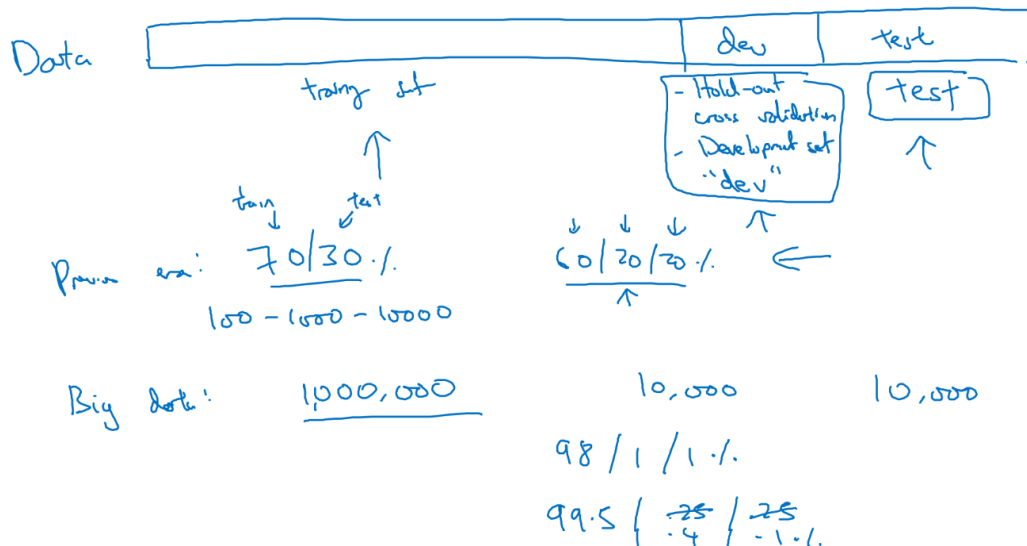
Applied ML is An Iterative Process

- When starting a new application, it's almost impossible to correctly guess the right values for your hyperparameters on the first attempt
- Write code to run an experiment of several values and get back a result that tells you how well the particular network configuration works, then refine ideas and change choices



Data Subsets

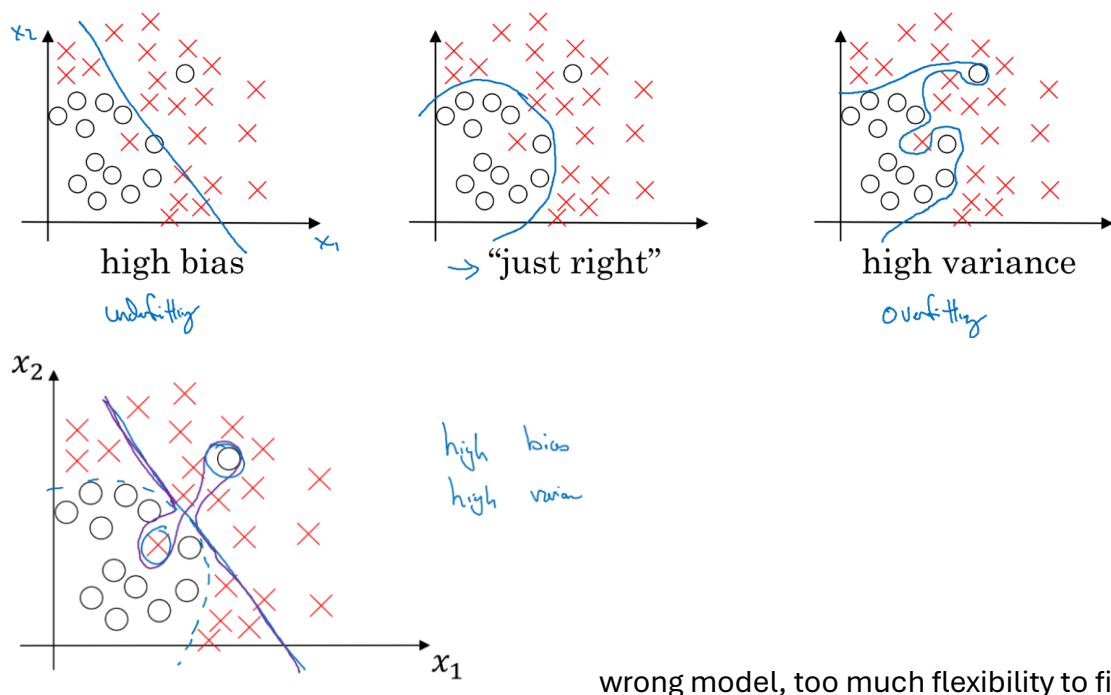
- **Training set:** train algorithms
- **Cross-validation/development/dev set:** see which model performs best
- **Test set:** evaluate best model (unbiased estimate of how well the algorithm is doing)
- With larger data, don't use the 60-20-20 rule. Dev and test should be much smaller



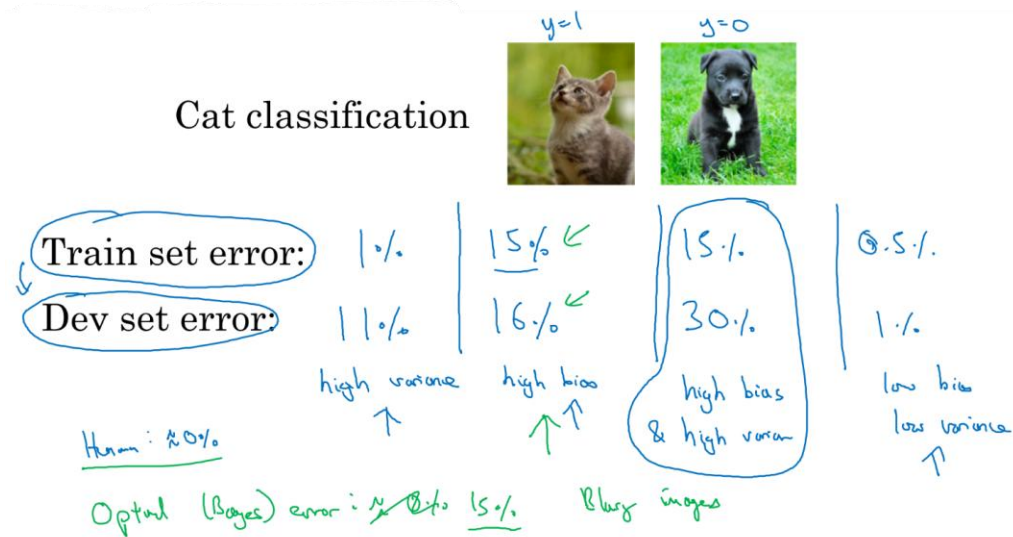
- Mismatched train/test distribution
 - Make sure dev and test come from the same distribution
 - Example: training set pictures from webpages and dev/test sets from app users
 - Not having a test set (only dev set) might be okay if you don't need an unbiased estimate of performance
 - “train/test” → really a train/dev set because they're overfitting to the test set

Bias and Variance

- High bias = underfitting
- High variance = overfitting

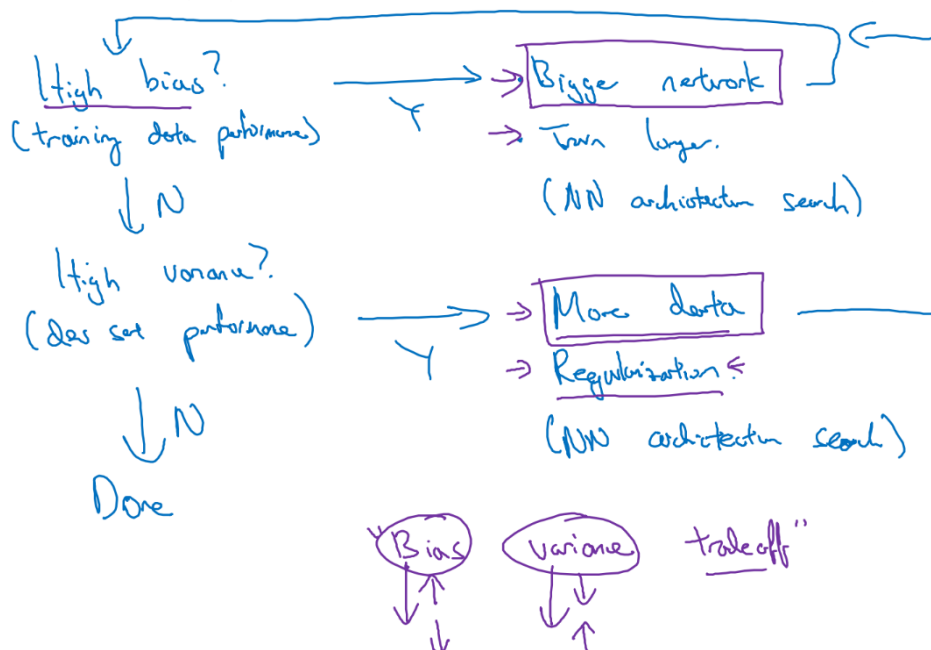


- Example:
 - Train set error low, dev set error high = high variance → overfit
 - Train set error high, dev set error low = high bias → underfit
 - Train set error high, dev set error high = high bias and high variance
 - Ideal: Train set error low, dev set error low = low bias and low variance → good fit
 - Bayes (optimal) error should be considered, because some amount of error is reasonable and a model is performing better than you would think when comparing to zero



Basic Recipe for Machine Learning

- High bias (training data performance) → bigger network (more hidden layers or hidden units), train longer, more advanced algorithms, NN architecture search
 - Getting more training data won't help here
- High variance (dev set performance) → more data, regularization, NN architecture
- In modern deep learning, if you can get more data then getting a bigger network almost always reduces bias without hurting variance (bypass historical bias-variance tradeoff)



Regularization

- Prevents overfitting & reduces variance
- λ **regularization parameter** applied to w term (can also apply to b , but doesn't make much difference)
- For logistic regression:

$\min_{w,b} J(w,b)$
 $w \in \mathbb{R}^{n_x}, b \in \mathbb{R}$
 $\lambda =$ regularization parameter

$J(w,b) = \frac{1}{m} \sum_{i=1}^m \ell(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|_2^2$
 λ lambda
 λ lambda

L_2 regularization $\|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w \leftarrow$
 ~~$\frac{\lambda}{2m} b^2$~~
omit

L_1 regularization $\frac{\lambda}{2m} \sum_{j=1}^{n_x} |w_j| = \frac{\lambda}{2m} \|w\|_1$
 w will be sparse

- L1 (abs value): w will be sparse (have a lot of 0s in it). Can help compress the model (some of the parameters are zero, less memory)
- L2 ("weight decay") is used more

$\rightarrow J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \frac{1}{m} \sum_{i=1}^m \ell(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^{[l]}\|_F^2$

$\|w^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (w_{ij}^{[l]})^2$
 $w^{[l]}: (n^{[l]}, n^{[l-1]})$

"Frobenius norm"
 $\| \cdot \|_2^2$
 $\| \cdot \|_F^2$

$dW^{[l]} = \left[\text{(from backprop)} + \frac{\lambda}{m} W^{[l]} \right]$
 $\frac{\partial J}{\partial w^{[l]}} = dW^{[l]}$

$\rightarrow W^{[l]} := W^{[l]} - \alpha dW^{[l]}$

"Weight decay"
 $W^{[l]} := W^{[l]} - \alpha \left[\text{(from backprop)} + \frac{\lambda}{m} W^{[l]} \right]$

$= W^{[l]} - \frac{\alpha \lambda}{m} W^{[l]} - \alpha \text{(from backprop)}$

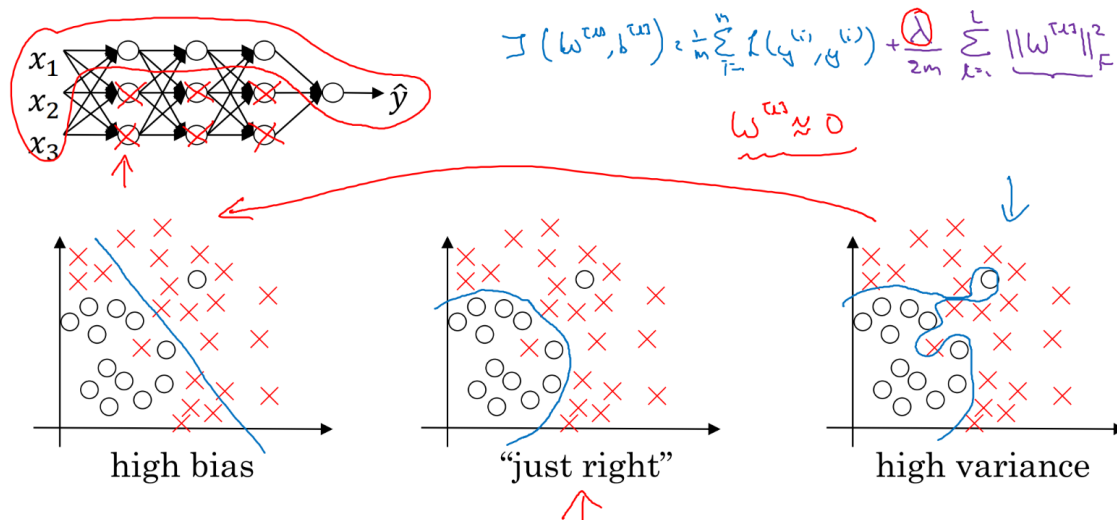
$= \underbrace{\left(1 - \frac{\alpha \lambda}{m}\right)}_{< 1} W^{[l]} - \alpha \text{(from backprop)}$

$$\|w^{[l]}\|^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (w_{i,j}^{[l]})^2$$

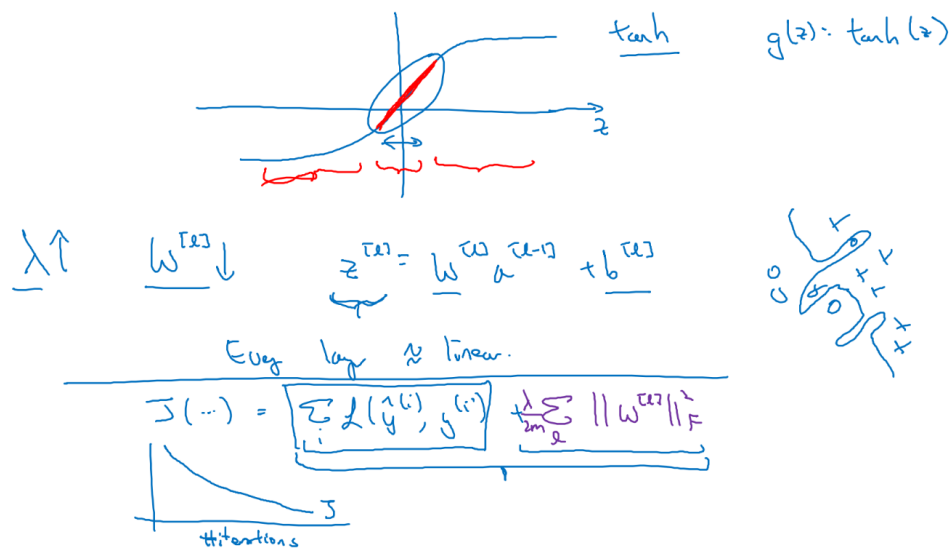
- **Frobenius norm:**
 - Sum of squares of elements of a matrix
 - rows i of the matrix are the number of neurons in the current layer $n^{[l]}$
 - columns j of the weight matrix equal the number of neurons in the previous layer $n^{[l-1]}$
- Add regularization term to dW and compute update same as before

Why Regularization Prevents Overfitting

- Regularization term penalizes the weight matrices from being too large
- Why does shrinking the L2/Frobenius norm with the parameters cause less overfitting?
- Really large λ really incentivizes the weight matrices to be close to 0. Maybe it'll set the weights so close to zero for a lot of hidden units that it's basically zeroing out a lot of the impact of these hidden units. NN becomes simplified and smaller
- Hopefully an intermediate λ exists to leave enough complexity of hidden units but still control overfitting



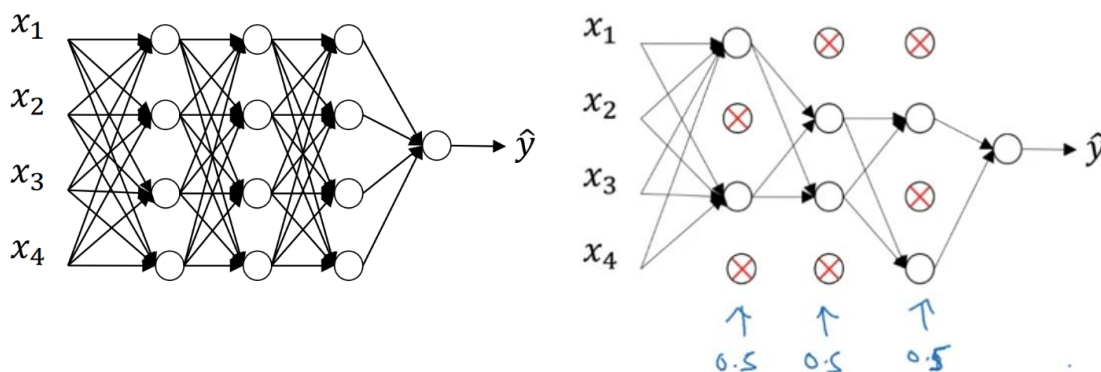
- For tanh,
 - Large λ = small parameters (penalized) = small $z = g(z)$ every layer roughly linear



Monotonic decrease in plot of cost J vs iterations when using the regularization term

Dropout Regularization

- Go through each layer and set some probability of eliminating a node, remove all ingoing/outgoing links from those nodes, backprop train on diminished network. Repeat random elimination of nodes and training
- Training much smaller network on each example



- Implementing dropout ("inverted dropout")
 - Dropout vector $d^{[l]}$ (same shape as $a^{[l]}$) less than some threshold probability of keeping a given hidden unit
 - Generate a random matrix where for each example and each hidden unit there's a 0.8 chance that the corresponding $d^{[l]}$ will be 1, and 0.2 chance of 0
 - Take activations from layer l and set equal to old activations $a^{[l]}$ times (element-wise) dropout matrix
 - Every element of $d^{[l]}$ equal to zero (FALSE) drops out
 - *Scale $a^{[l]}$ up: divide by keep probability (to keep the EV of $z^{[l]}$ and $a^{[l]}$ the same)

Illustrate with layer $l=3$. keep-prob = $\frac{0.8}{x}$ 0.2

→ $d3 = \text{np.random.rand}(a3.\text{shape}[0], a3.\text{shape}[1]) < \text{keep-prob}$

$a3 = \text{np.multiply}(a3, d3)$ # $a3 \times d3$

→ $a3 /= \text{keep-prob}$ ←

50 units. → 10 units shut off

$z^{[4]} = w^{[4]} \cdot a^{[3]} + b^{[4]}$

↑ $\nwarrow$ reduced by 20% Test

$/= 0.8$

- Making predictions at test time
 - Do not use dropout explicitly; do not want output to be random
 - No extra scaling parameter needed here if you scaled up during training
 - You could run a prediction process many times with different hidden units randomly dropped out and average across them, but computationally inefficient

$a^{[0]} = X$

No drop out.

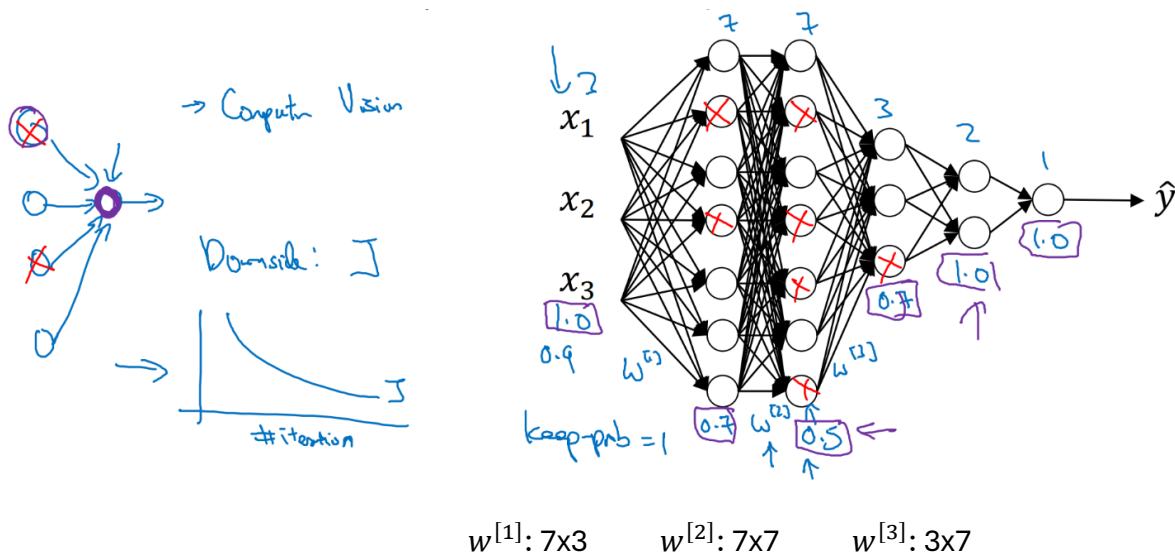
$$\begin{aligned}
 z^{[1]} &= w^{[1]} a^{[0]} + b^{[1]} \\
 a^{[1]} &= g^{[1]}(z^{[1]}) \\
 z^{[2]} &= w^{[2]} a^{[1]} + b^{[2]} \\
 a^{[2]} &= \dots
 \end{aligned}$$

↓
y

$/= \text{keep-prob}$

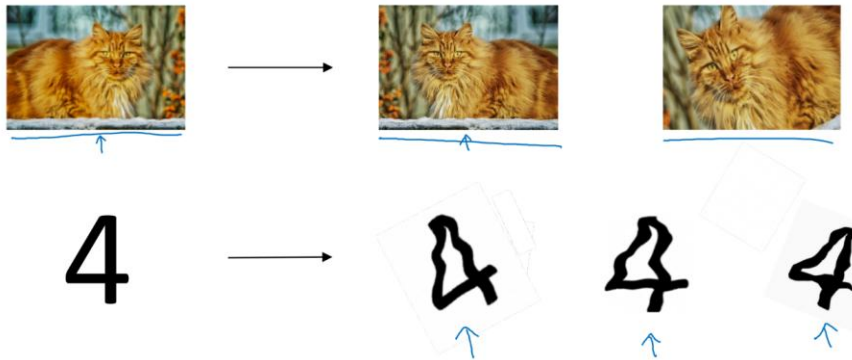
Why Dropout Works

- Intuition: Can't rely on any one feature, so have to spread out weights → shrink squared norm of weights (similar effect as L2)
- Dropout is an adaptive form of L2 regularization, but the L2 penalty on different weights is different depending on the size of the activations being multiplied that way
- If you're more worried about overfitting in some layers than others (e.g. larger weight matrices), you can have a lower keep prob than the other layers. However, this introduces hyperparameters to tune. You could have one hyperparameter for which layers to apply dropout to and which not to
- In general, the number of neurons in the previous layer gives us the number of columns of the weight matrix, and the number of neurons in the current layer gives us the number of rows in the weight matrix
- You lose the debugging tool of cost function plotting when introducing dropout. Check the code itself before implementing dropout

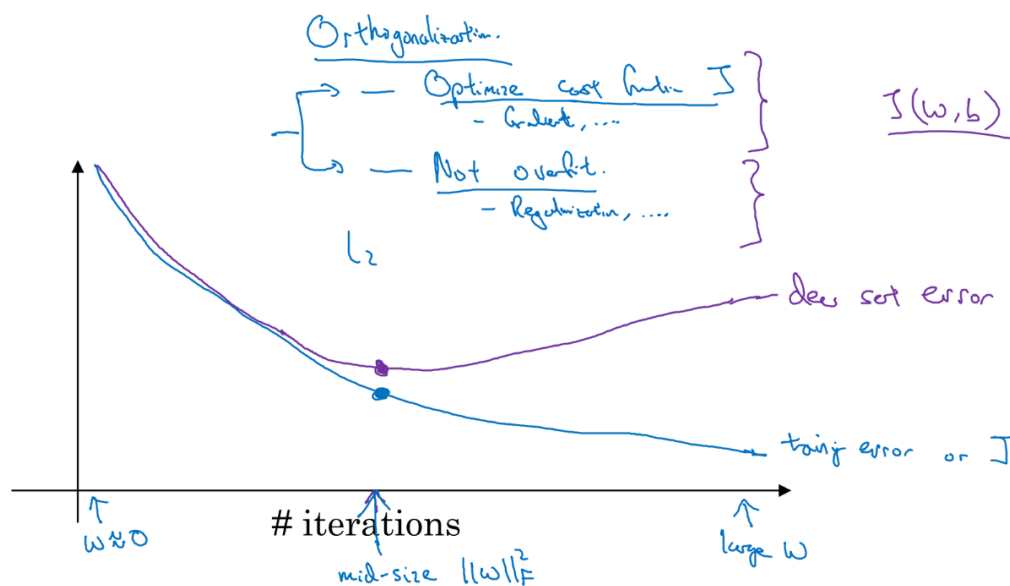


Other Regularization Methods

- Data augmentation
 - Synthesize training data by distorting (flipping, cropping, transforming) existing training data into new training examples
 - Not as much information added as new data (slightly redundancy), but works for budget constraints

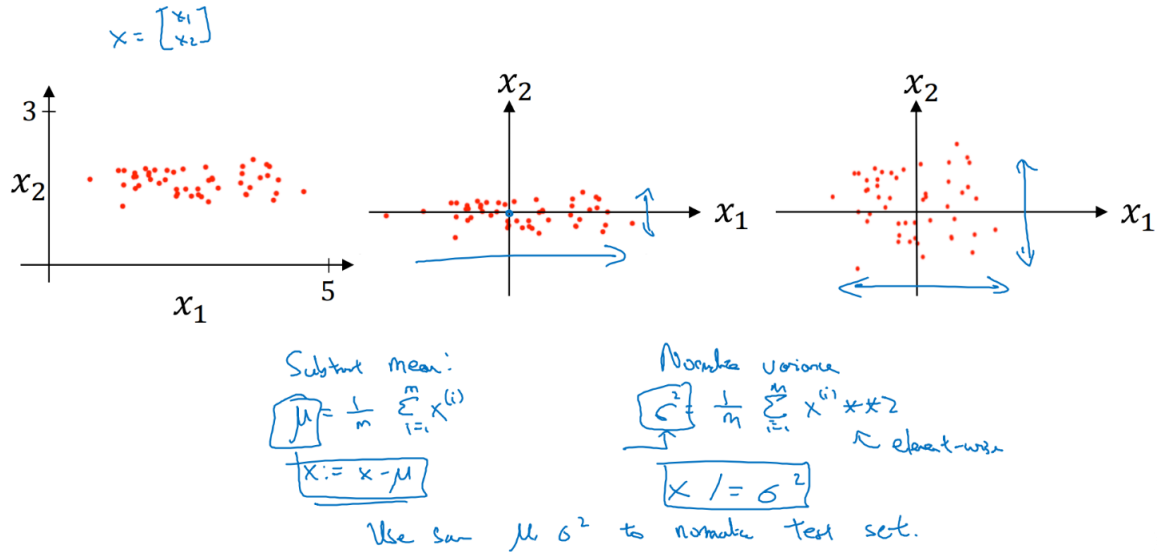


- Early stopping
 - As you run gradient descent, plot training error or cost function and dev set error. When dev set stops going down and start going up, stop training the NN and take the values that achieved that dev set error
 - NN is prevented from fitting so much extra to reduce training error
 - Orthogonalization: separate tools for optimizing J and preventing overfitting
 - Early stopping couples these two so they can't be addressed independently
 - L2 regularization makes hyperparameter space search easier to decompose, but may have to try a lot of values of λ à computationally expensive

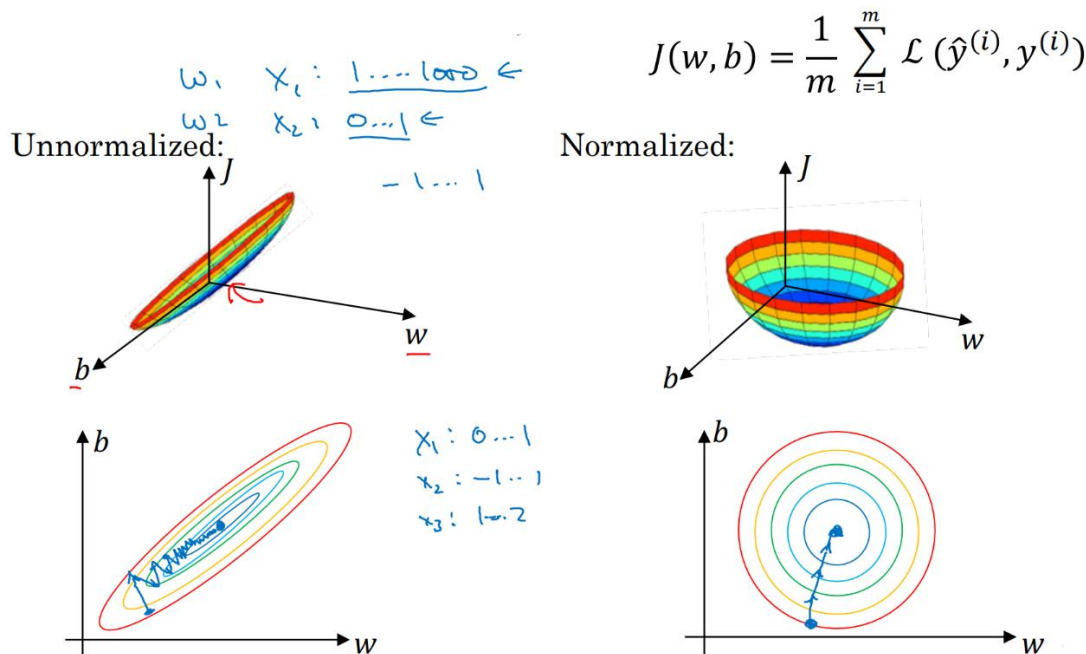


Normalizing Inputs

- Subtract out mean and normalize variances
- Use the same mean and variance (calculated on the training set) to normalize test set

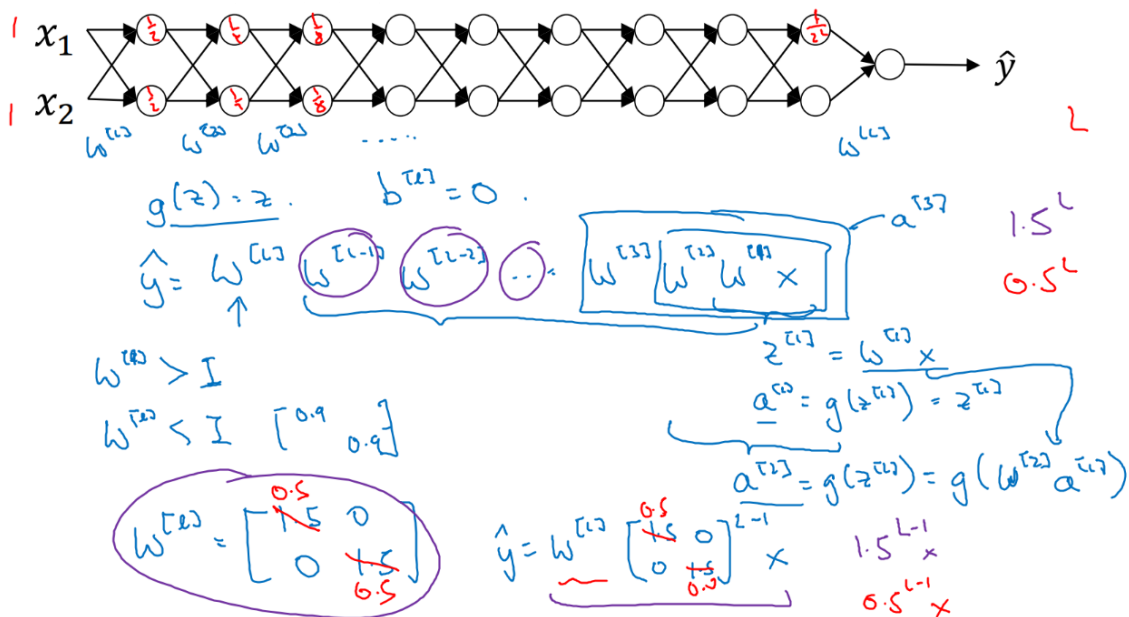


- Unnormalized features $\rightarrow$ elongated cost function $\rightarrow$ gradient descent needs lots of steps (smaller learning rate)
- Normalized features $\rightarrow$ bigger steps for gradient descent (faster)

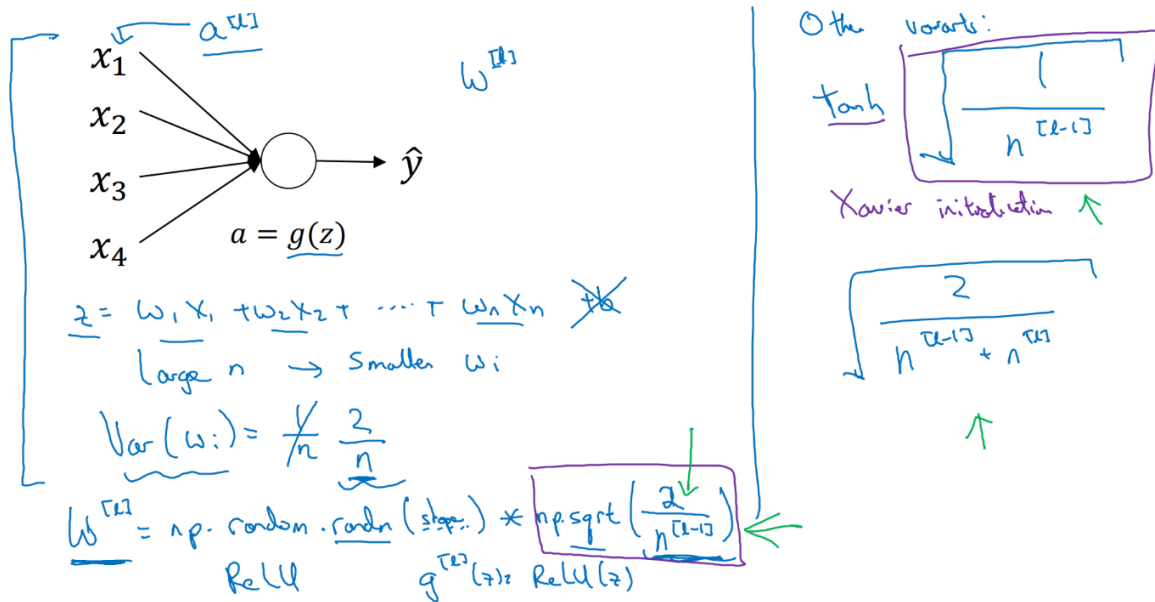


Vanishing/Exploding Gradients

- Derivatives can get very big (exploding) or small (vanishing), which makes training difficult
- Careful choices of random weight initialization significantly reduce this problem
- If $W^{[L]} > I$ (>1), with a deep network the activations will explode
- If $W^{[L]} < I$ (<1), with a deep network the activations will vanish (decrease exponentially)
- The derivatives computed will also increase/decrease exponentially as a function of the number of layers

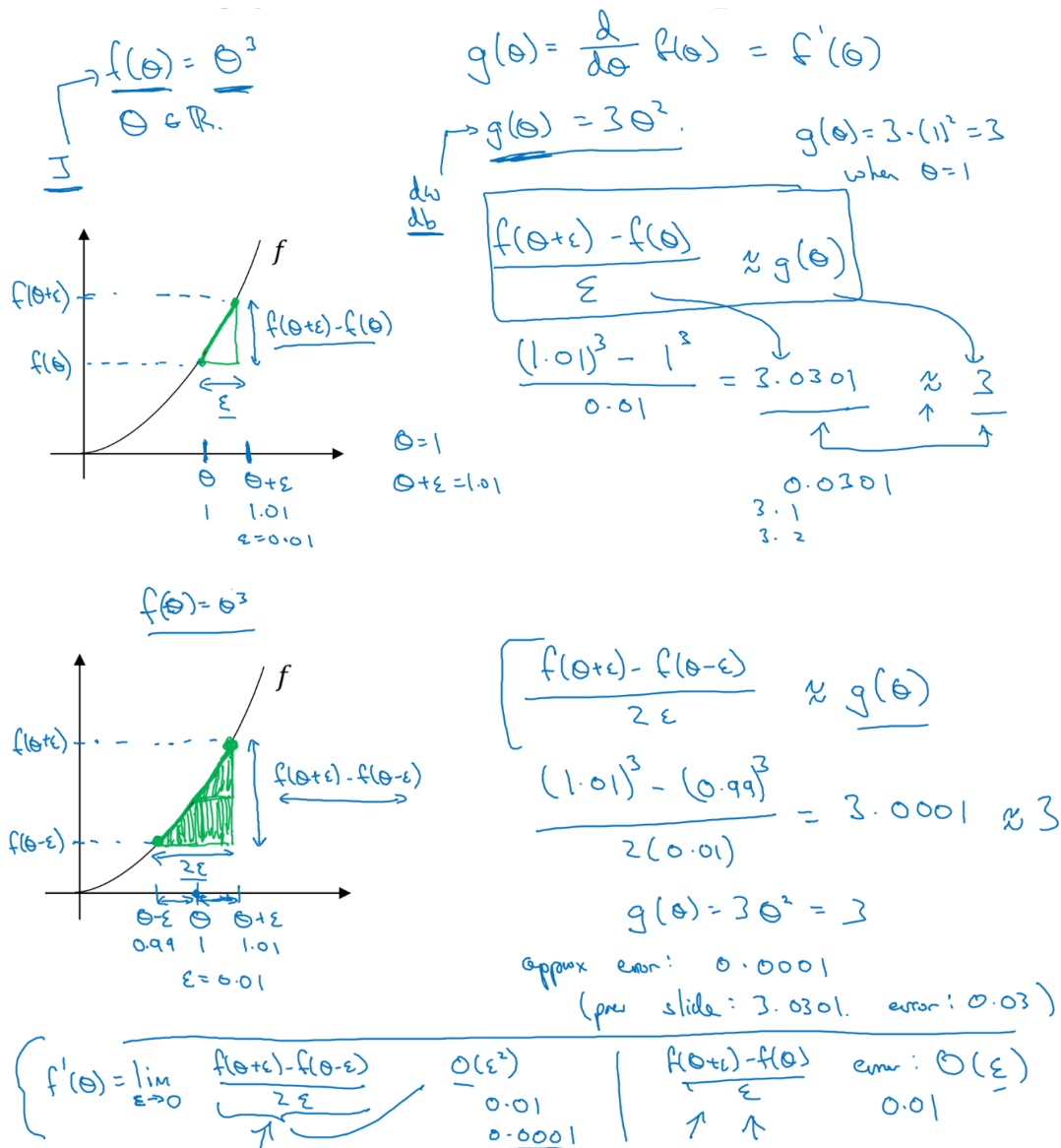


- Weight initialization - single neuron example
 - Large $n \rightarrow$ smaller we want w_i to be $\rightarrow$ Set $\text{var}(W) = 1/n \rightarrow \text{np.sqrt}(1/n^{[l-1]})$
 - ReLU: $\text{var}(W) = 2/n \rightarrow \text{np.sqrt}(2/n^{[l-1]})$
 - If input features/activations are roughly mean 0 and standard variance, then z will be as well
 - Helps reduce the exploding/vanishing gradients problem by setting the weight matrices w to something that isn't too much greater or less than 1



Numerical Approximation of Gradients

- Checking derivative computation
 - On the order of ε^2 (two-sided difference, bottom) much more accurate than on the order of ε (one-sided difference, top)



Gradient Checking

- Debug backprop implementation
- Reshape $W^{[1]}, b^{[1]}, \dots, W^{[L]}, b^{[L]}$ and concatenate into a big vector θ
 - $J(W^{[1]}, b^{[1]}, \dots, W^{[L]}, b^{[L]}) = J(\theta)$
- Reshape $dW^{[1]}, db^{[1]}, \dots, dW^{[L]}, db^{[L]}$ and concatenate into a big vector $d\theta$
- Derivatives have same dimensions
- Is $d\theta$ the gradient of $J(\theta)$?
- For each component of θ , compute the approximate derivative $d\theta_{\text{approx}}$ using the two-sided difference. Nudge θ_i by adding ε (and subtracting on the other side). Should be approximately equal to $d\theta_i$ (the partial derivative of cost function J with respect to θ_i).
- $d\theta_{\text{approx}}$ has same dimension as $d\theta$ and θ . Check if $d\theta_{\text{approx}} \approx d\theta$ by calculating the Euclidean distance between the two vectors: $\|d\theta_{\text{approx}} - d\theta\|_2 / (\|d\theta_{\text{approx}}\| + \|d\theta\|)$
- Any bigger than 10^{-3} is a bug concern

$J(\theta) = J(\theta_1, \theta_2, \dots)$

for each i :

$$\rightarrow \underline{d\theta_{\text{approx}}[i]} = \frac{J(\theta_1, \theta_2, \dots, \overset{\downarrow}{\theta_i + \varepsilon}, \dots) - J(\theta_1, \theta_2, \dots, \overset{\downarrow}{\theta_i - \varepsilon}, \dots)}{2\varepsilon}$$

$$\approx \underline{d\theta[i]} = \frac{\partial J}{\partial \theta_i} \quad \Bigg| \quad d\theta_{\text{approx}} \stackrel{?}{\approx} d\theta$$

Check

$$\rightarrow \frac{\|d\theta_{\text{approx}} - d\theta\|_2}{\|d\theta_{\text{approx}}\|_2 + \|d\theta\|_2}$$

$\varepsilon = 10^{-7}$

$$\approx \frac{10^{-7}}{10^{-5}} - \text{great!} \leftarrow$$

$$\rightarrow 10^{-3} - \text{worry.} \leftarrow$$

Gradient Check Implementation Tips

- Don't use in training – only to debug
 - Computing $d\theta_{\text{approx}}^{[i]}$ for each i is very slow
- If algorithm fails grad check, look at components to try to identify bug
 - If values with θ far off all correspond to $db^{[i]} \rightarrow$ bug in computing db

$$\begin{array}{ccc} d\theta_{\text{approx}}^{[i]} & \longleftrightarrow & d\theta^{[i]} \\ \uparrow \quad \uparrow & & \uparrow \\ db^{[i]} & & dw^{[i]} \end{array}$$

- Remember regularization

$$\underline{J(\theta)} = \frac{1}{n} \sum_i \ell(y^{(i)}, \hat{y}^{(i)}) + \frac{\lambda}{2n} \sum_i \|w^{(i)}\|_2^2$$

$d\theta = \text{gradient of } J \text{ wrt. } \theta$

- Doesn't work with dropout
 - Dropout optimizes a cost function J defined by summing over all exponentially large subsets of nodes they could eliminate in any iteration. VERY difficult, and is just sampling the cost function each time
 - Turn on dropout later

$$\underline{J} \quad \underline{\text{keep-prob} = 1.0}$$

- Run at random initialization; perhaps again after some training
 - Maybe the implementation of GD is correct when $w, b \approx 0$, but gets more inaccurate as w, b become large

If you have 10,000 examples, how would you split the train/dev/test set? Choose the best option.

98% train. 1% dev. 1% test.

[not big enough for this approach]

33% train. 33% dev. 33% test.

60% train. 20% dev. 20% test.

In a personal experiment, an M.L. student decides to not use a test set, only train-dev sets. In this case which of the following is true?

Not having a test set is unacceptable under any circumstance.

He won't be able to measure the bias of the model.

He won't be able to measure the variance of the model.

He might be overfitting to the dev set.

A model developed for a project is presenting high bias. One of the sponsors of the project offers some resources that might help reduce the bias. Which of the following additional resources has a better chance to help reduce the bias?

Use different sources to gather data and better test the model.

Give access to more computational resources like GPUs.

Gather more data for the project.

Your classifier for bananas and oranges gets a training set error of 0.1% and a development set error of 11%.

Which of the following statements are true? (Choose all that apply.)

The model is overfitting the training set.

The model has a high variance.

The model has a very high bias.

The model is overfitting the development set.

You are working on an automated check-out kiosk for a supermarket and are building a classifier for apples, bananas, and oranges. Suppose your classifier obtains a training set error of 19% and a development set error of 21%. Which of the following is the most promising strategy to improve your classifier? (Assume the human error is approximately 0%).

Increase the regularization parameter λ .

Use a bigger network. [low variance; only a difference of 2%]

Get more training data.

What is weight decay?

The process of gradually decreasing the learning rate during training.

Gradual corruption of the weights in the neural network if it is trained on noisy data.

A regularization technique (such as L2 regularization) that results in gradient descent shrinking the weights on every iteration.

A technique to avoid vanishing gradient by imposing a ceiling on the values of the weights.

Which of the following are regularization techniques?

Increase the number of layers of the network.

Dropout.

Weight decay.

Gradient Checking.

True or False: To reduce high variance, the regularization hyperparameter λ must be increased.

False

True

True or False: In L2 regularization, the lambda hyperparameter directly influences the calculations used by the model to make predictions during testing.

False

True

Which of the following are true about dropout? (Choose all that apply.)

It helps to reduce the variance of a model.

It helps to reduce the bias of a model.

In practice, it eliminates units of each layer with a probability of $1 - \text{keep_prob}$.

In practice, it eliminates units of each layer with a probability of keep_prob .

With the inverted dropout technique, at test time:

You do not apply dropout (do not randomly eliminate units), but keep the $1/\text{keep_prob}$ factor in the calculations used in training.

You apply dropout (randomly eliminating units) but keep the $1/\text{keep_prob}$ factor in the calculations used in training.

You apply dropout (randomly eliminating units) and do not keep the $1/\text{keep_prob}$ factor in the calculations used in training

You do not apply dropout (do not randomly eliminate units) and do not keep the $1/\text{keep_prob}$ factor in the calculations used in training

Decreasing the parameter keep_prob from (say) 0.6 to 0.4 will likely cause the following:

Increasing the regularization effect.

Reducing the regularization effect.

Causing the neural network to have a higher variance.

During training a deep neural network that uses the tanh activation function, the value of the gradients is practically zero. Which of the following is most likely to help the vanishing gradient problem?

Increase the number of cycles during the training.

Use Xavier initialization.

Use a larger regularization parameter.

Increase the number of layers of the network.

Which of these techniques are useful for reducing variance (reducing overfitting)? (Choose all that apply.)

Vanishing gradient

Gradient Checking

Data augmentation

Exploding gradient

Xavier initialization

Dropout

L2 regularization

Which of the following actions increase the regularization of a model? (Choose all that apply)

Make use of data augmentation.

Decrease the value of the hyperparameter lambda.

Increase the value of the hyperparameter lambda.

Increase the value of keep_prob in dropout.

Normalizing the data.

Suppose that a model uses, as one feature, the total number of kilometers walked by a person during a year, and another feature is the height of the person in meters. What is the most likely effect of normalization of the input data?

It will make the data easier to visualize.

It won't have any positive or negative effects.

It will make the training faster.

It will increase the variance of the model.

Which of the following is the correct expression to normalize the input x ?

$$x = \frac{x}{\sigma}$$

$$x = \frac{1}{m} \sum_{i=1}^m (x^{(i)})^2$$

$$x = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$x = \frac{x - \mu}{\sigma}$$

Mini-Batch Gradient Descent

- Vectorization allows you to efficiently compute on m examples, but if m is large it can still be slow
- Batch vs mini batch gradient descent
 - **Batch:** process entire training set (X, Y) at the same time
 - **Mini-batch:** process a single mini-batch $(X^{(t)}, Y^{(t)})$ at a time

$$\begin{aligned}
 X &= \underbrace{\begin{bmatrix} x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(1000)} \end{bmatrix}}_{X^{(1)}} \underbrace{\begin{bmatrix} x^{(1001)} & \dots & x^{(2000)} \end{bmatrix}}_{X^{(2)}} \dots \underbrace{\begin{bmatrix} \dots & x^{(m)} \end{bmatrix}}_{X^{(5000)}} \\
 &\quad (n_x, m) \quad (n_x, 1000) \quad (n_x, 1000) \quad (n_x, 1000) \\
 Y &= \underbrace{\begin{bmatrix} y^{(1)} & y^{(2)} & y^{(3)} & \dots & y^{(1000)} \end{bmatrix}}_{Y^{(1)}} \underbrace{\begin{bmatrix} y^{(1001)} & \dots & y^{(2000)} \end{bmatrix}}_{Y^{(2)}} \dots \underbrace{\begin{bmatrix} \dots & y^{(m)} \end{bmatrix}}_{Y^{(5000)}} \\
 &\quad (1, m) \quad (1, 1000) \quad (1, 1000) \quad (1, 1000)
 \end{aligned}$$

What if $m = 5,000,000$?

5,000 mini-batches of 1,000 each

Mini-batch t : $X^{(t)}, Y^{(t)}$

$$\begin{array}{c}
 x^{(i)} \\
 z^{[l]} \\
 x^{(t)}, y^{(t)}
 \end{array}$$

() = training example, [] = layer of NN, {} = mini-batch

- Mini-batch
 - For each batch, run gradient descent (**epoch**): forward prop, compute cost function J , implement backprop to compute gradients wrt $J^{(t)}$ and update weights

Repeat $\{$ for $t = 1, \dots, 5000 \}$

Forward prop on $X^{(t)}$.

$$\begin{aligned}
 z^{(1)} &= W^{(0)} X^{(t)} + b^{(1)} \\
 A^{(1)} &= g^{(1)}(z^{(1)}) \\
 &\vdots \\
 A^{(L)} &= g^{(L)}(z^{(L)})
 \end{aligned}$$

Vectorized implementation (1000 examples)

Compute cost $J = \frac{1}{1000} \sum_{i=1}^L \mathcal{L}(y^{(i)}, \hat{y}^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_{l=1}^L \|W^{(l)}\|_F^2$

Backprop to compute gradients wrt $J^{(t)}$ (using $(X^{(t)}, Y^{(t)})$)

$$W^{(l)} = W^{(l)} - \alpha \frac{\partial J}{\partial W^{(l)}}, \quad b^{(l)} = b^{(l)} - \alpha \frac{\partial J}{\partial b^{(l)}}$$

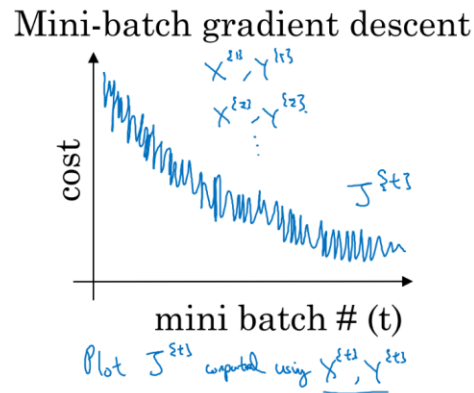
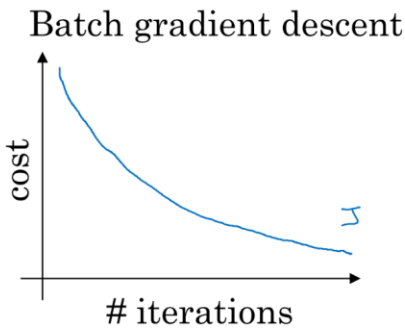
"1 epoch" pass through training set.

1 step of gradient descent using $X^{(t)}, Y^{(t)}$ (as if $m=1000$)

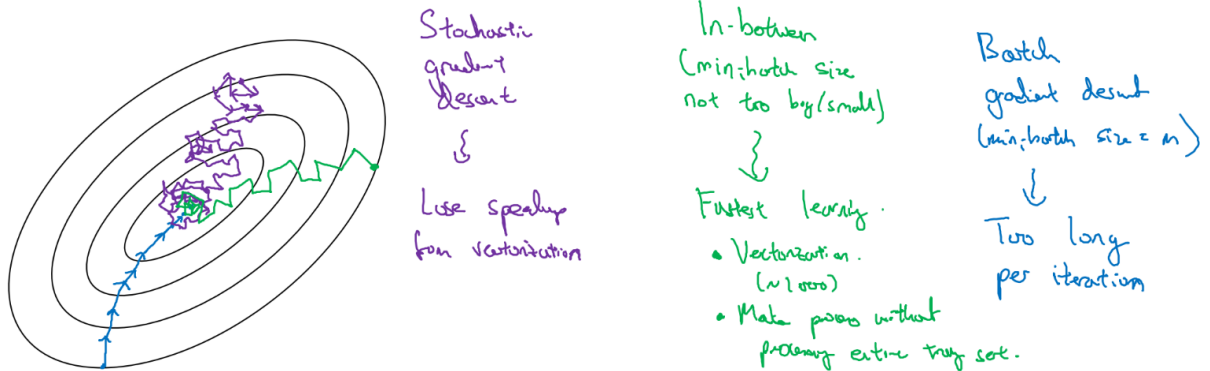
X, Y

Here L = number of samples in the mini-batch

- Training with mini batch gradient descent
 - Cost function J may not decrease every iteration, because it's as if you're training on a different training set, but should still trend downwards



- Choosing mini-batch size
 - If mini-batch size = m : batch gradient descent ($X^{(1)}, Y^{(1)} = (X, Y)$)
 - Low noise, large steps, but takes a long time processing the whole training set
 - If mini-batch size = 1: stochastic gradient descent [every example is its own ($X^{(1)}, Y^{(1)} = (x^{(1)}, y^{(1)}) \dots (x^{(2)}, y^{(2)})$ mini-batch)]
 - Noisy, does not converge, and loses speed-up from vectorization, but progress after processing one example
 - In practice, somewhere between 1 and m
 - Fastest learning: vectorization and progress without processing the whole training set

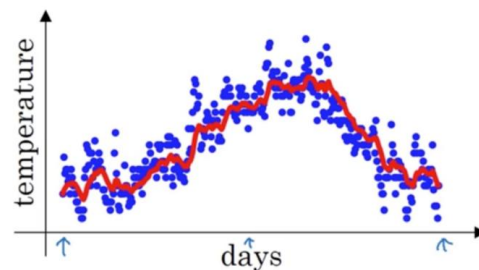


- If small training set ($m \leq 2000$): use batch gradient descent
- Typical mini-batch sizes: 64 (2^6), 128 (2^7), 256 (2^8), 512 (2^9), 1024 (2^{10}) (power of 2 faster)
- Make sure minibatch $X^{(t)}, Y^{(t)}$ fits in CPU/GPU memory

Exponentially Weighted (Moving) Averages

- Tool for building optimization algorithms
- Example: London temperature
 - Take weighted average each day. Only some of the new day's information gets added in

$$\begin{aligned}
 \theta_1 &= 40^\circ\text{F} & 4^\circ\text{C} & \leftarrow \\
 \theta_2 &= 49^\circ\text{F} & 9^\circ\text{C} & \\
 \theta_3 &= 45^\circ\text{F} & & \\
 &\vdots & & \\
 \theta_{180} &= 60^\circ\text{F} & 15^\circ\text{C} & \\
 \theta_{181} &= 56^\circ\text{F} & & \\
 &\vdots & &
 \end{aligned}$$



$$\begin{aligned}
 v_0 &= 0 \\
 v_1 &= 0.9 v_0 + 0.1 \theta_1 \\
 v_2 &= 0.9 v_1 + 0.1 \theta_2 \\
 v_3 &= 0.9 v_2 + 0.1 \theta_3 \\
 &\vdots \\
 v_t &= 0.9 v_{t-1} + 0.1 \theta_t
 \end{aligned}$$

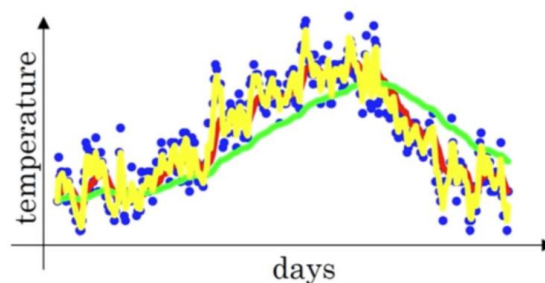
$$v_t = \beta v_{t-1} + (1 - \beta) \theta_t$$

- $v_t = \beta v_{t-1} + (1 - \beta) \theta_t$ approximately averages over $1/(1-\beta)$ days temperature
- $\beta=0.9$ averages over 10 days temperature
- $\beta=0.98$ averages over 50 days temp $\rightarrow$ curve is smoother, but shifted further to the right (adapts more slowly when temperature changes)
- $\beta=0.5$ averages over 2 days temp $\rightarrow$ much more noisy/susceptible to outlier, but adapts much quicker

$$\begin{aligned}
 v_t &= \beta v_{t-1} + (1 - \beta) \theta_t \leftarrow \\
 \beta &= 0.9 & : & \approx 10 \text{ days' temp.} \\
 \beta &= 0.98 & : & \approx 50 \text{ days} \\
 \beta &= 0.5 & : & \approx 2 \text{ days}
 \end{aligned}$$

v_t is approximately
 average over
 $\rightarrow \approx \frac{1}{1-\beta}$ days'
 temperature.

$$\frac{1}{1-0.98} = 50$$



- Intuition:

- Exponential decay of weighted sum
- Coefficients added up to (close to) 1 [bias correction]
- $\beta^{1/\varepsilon} = (1 - \varepsilon)^{1/\varepsilon} = 1/\varepsilon \rightarrow 1/(1 - \beta), \varepsilon = 1 - \beta$

$$v_t = \beta v_{t-1} + (1 - \beta) \theta_t$$

$$v_{100} = 0.9v_{99} + 0.1\theta_{100}$$

$$v_{99} = 0.9v_{98} + 0.1\theta_{99}$$

$$v_{98} = 0.9v_{97} + 0.1\theta_{98}$$

$$\dots$$

$$\rightarrow v_{100} = 0.1\theta_{100} + 0.9v_{99} = 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9v_{98})$$

$$= 0.1\theta_{100} + 0.1 \times 0.9 \cdot \theta_{99} + 0.1(0.9)^2\theta_{98} + 0.1(0.9)^3\theta_{97} + 0.1(0.9)^4\theta_{96} + \dots$$

$$0.9^{10} \approx 0.35 \approx \frac{1}{e}$$

$$\frac{(1-\varepsilon)^{1/\varepsilon}}{\varepsilon} \approx \frac{1}{\varepsilon}$$

$$\varepsilon = 0.02 \rightarrow 0.98^{50} \approx \frac{1}{e}$$

- Implementing

- Initialize $v_0 = 0$, then on each day get next θ_t and update v_t as $\beta v_t + (1 - \beta) \theta_t$
- Take very little memory, only need to keep one real number in memory
- Efficient, but slightly less accurate

$$v_0 = 0$$

$$v_1 = \beta v_0 + (1 - \beta) \theta_1$$

$$v_2 = \beta v_1 + (1 - \beta) \theta_2$$

$$v_3 = \beta v_2 + (1 - \beta) \theta_3$$

$$\dots$$

$$v_0 := 0$$

$$v_0 := \beta v + (1 - \beta) \theta_1$$

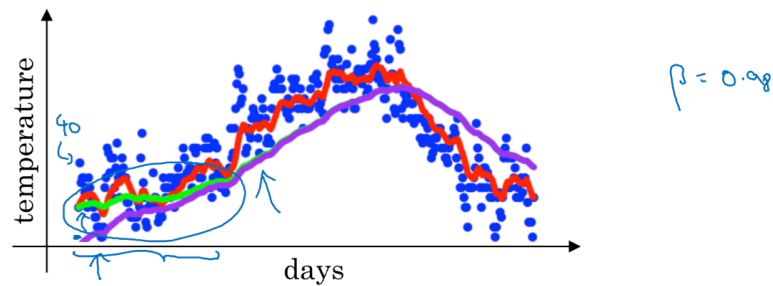
$$v_0 := \beta v + (1 - \beta) \theta_2$$

$$\vdots$$

$\rightarrow v_0 = 0$
 Repeat {
 Get next θ_t
 $v_0 := \beta v_0 + (1 - \beta) \theta_t \leftarrow$
 }

Bias Correction in Exponentially Weighted Average

- Makes computation of exponentially weighted averages more accurate, especially in the initial phase
- $V_t / (1 - \beta^t) \rightarrow \beta v_{t-1} + (1 - \beta) \theta_t / (1 - \beta^t)$
- For $t=2$: becomes a weighted average of θ_2 and θ_1 , removing bias
- As t becomes large, $1 - \beta^t$ approaches 0 $\rightarrow$ bias correction makes no difference



$$\rightarrow v_t = \beta v_{t-1} + (1 - \beta) \theta_t$$

$$v_0 = 0$$

$$v_1 = 0.98 v_0 + 0.02 \theta_1$$

$$\begin{aligned} v_2 &= 0.98 v_1 + 0.02 \theta_2 \\ &= 0.98 \times 0.02 \times \theta_1 + 0.02 \theta_2 \\ &= 0.0196 \theta_1 + 0.02 \theta_2 \end{aligned}$$

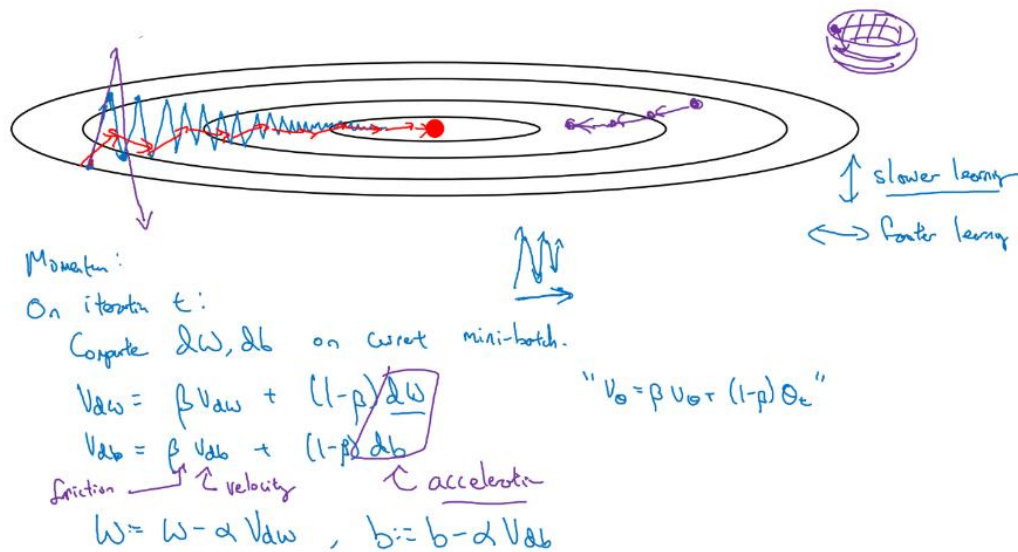
$$\frac{v_t}{1 - \beta^t}$$

$$t=2: 1 - \beta^t = 1 - (0.98)^2 = 0.0396$$

$$\frac{v_2}{0.0396} = \frac{0.0196 \theta_1 + 0.02 \theta_2}{0.0396}$$

Gradient Descent With Momentum

- Compute an exponentially weighted average of your gradients, and then use that gradient to update your weights instead
- Up and down oscillations slow down gradient descent and prevent you from using a larger learning rate.
- On the vertical axis, want slower learning to prevent oscillations
- On the horizontal axis, want faster learning to converge to global min faster
- Momentum:
 - On each iteration t , compute the derivatives dW and db on current min-batch (or whole batch), compute $v_{dw} = \beta v_{dw} + (1 - \beta) dW$ (moving average of derivatives for w) and $v_{db} = \beta v_{db} + (1 - \beta) db$, then update the parameters $W = W - \alpha v_{dw}$ and $b = b - \alpha v_{db}$. This smooths out the steps of gradient descent
 - Derivatives in vertical direction will waver but average around 0, derivatives in horizontal direction will point in the same direction (so be pretty big)
 - Ball roll analogy: β is friction, v_{dw} is velocity, derivatives are acceleration



- Implementation

- Steps: On iteration t :

- Compute dW, db on the current mini-batch

- $v_{dw} = \beta v_{dw} + (1-\beta) dW \rightarrow \beta v_{dw} + dW$

- $v_{db} = \beta v_{db} + (1-\beta) db \rightarrow v_{db} = \beta v_{db} + (1-\beta) db$

- $W = W - \alpha v_{dw}, b = b - \alpha v_{db}$

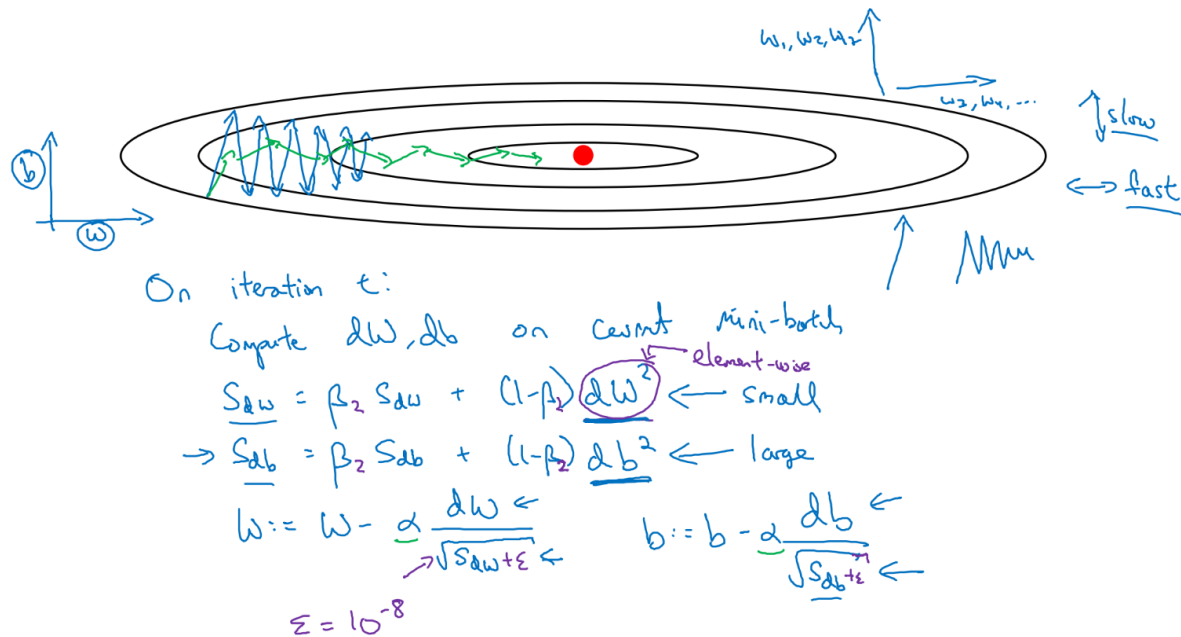
- Two hyperparameters: learning rate α (if using reduce eqns, α adjusts) and control of exp weighted avg β (commonly 0.9)

- Using reduced formulas affects scaling, resulting in needing to retune α

- Bias correction not done often, because after 10 iterations the moving average is usually warmed up is no longer a biased estimate

RMS (Root Mean Square) prop

- Steps: On iteration t : compute dW, db on current mini-batch, calculate $s_{dw} = \beta s_{dw} + (1-\beta)dW^2$ (element-wise square) and $s_{db} = \beta s_{db} + (1-\beta)db^2$, then update the parameters as $W := W - \alpha dw/\sqrt{s_{dw}}$ and $b := b - \alpha db/\sqrt{s_{db}}$
- Horizontal direction (ex. W) want to be fast $\rightarrow S_{dw}$ small, divide by small number
- Vertical direction (ex. b) want to be slow $\rightarrow S_{db}$ large, divide by large number $\rightarrow$ damp out oscillations
- Can use a larger learning rate α without diverging in the vertical direction
- Each “direction” can be some subset of parameters
- Make $\sqrt{s_{dw}}$ close to 0 doesn't make the parameter explode by adding a small ϵ to the denominator (“numerical stability”)
- Proposed in a Coursera course by Jeff Hinton, not an academic paper



Adam (Adaptive Moment Estimation) Optimization Algorithm

- Puts RMSprop and momentum together
- Steps: Initialize v_{dw} , s_{dw} , and v_{db} as 0. Then on iteration t : compute dW , db using current mini-batch (GD), then do the momentum exponentially weighted average update:

$$v_{dw} = \beta_1 v_{dw} + (1-\beta_1)dW \text{ and } v_{db} = \beta_1 v_{db} + (1-\beta_1)db,$$

then do RMSprop update as well:

$$s_{dw} = \beta_2 s_{dw} + (1-\beta_2)dW^2 \text{ and } s_{db} = \beta_2 s_{db} + (1-\beta_2)db^2,$$

then do bias correction: $v_{dw}^{\text{corrected}} = v_{dw} / (1-\beta_1^t)$ and $v_{db}^{\text{corrected}} = v_{db} / (1-\beta_1^t)$

$$s_{dw}^{\text{corrected}} = s_{dw} / (1-\beta_2^t) \text{ and } s_{db}^{\text{corrected}} = s_{db} / (1-\beta_2^t)$$

and finally update the parameters:

$$W := W - \alpha v_{dw}^{\text{corrected}} / \sqrt{s_{dw}^{\text{corrected}}} \text{ and } b := b - \alpha v_{db}^{\text{corrected}} / \sqrt{s_{db}^{\text{corrected}}}$$

$$v_{dw}=0, s_{dw}=0, v_{db}=0, s_{db}=0$$

On iteration t :

Compute dW, db using current mini-batch

$$v_{dw} = \beta_1 v_{dw} + (1-\beta_1)dW, \quad v_{db} = \beta_1 v_{db} + (1-\beta_1)db \quad \leftarrow \text{"momentum"} \beta_1$$

$$s_{dw} = \beta_2 s_{dw} + (1-\beta_2)dW^2, \quad s_{db} = \beta_2 s_{db} + (1-\beta_2)db^2 \quad \leftarrow \text{"RMSprop"} \beta_2$$

$yhat = np.array([.9, 0.2, 0.1, .4, .9])$

$$v_{dw}^{\text{corrected}} = v_{dw} / (1-\beta_1^t), \quad v_{db}^{\text{corrected}} = v_{db} / (1-\beta_1^t)$$

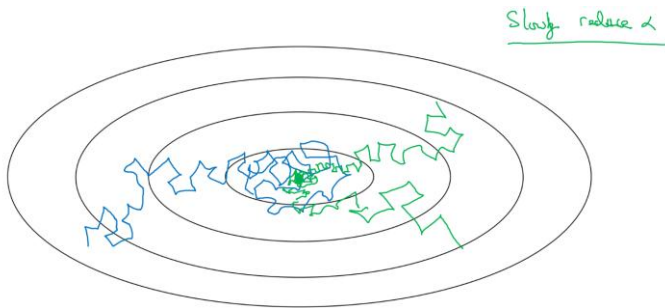
$$s_{dw}^{\text{corrected}} = s_{dw} / (1-\beta_2^t), \quad s_{db}^{\text{corrected}} = s_{db} / (1-\beta_2^t)$$

$$W := W - \alpha \frac{v_{dw}^{\text{corrected}}}{\sqrt{s_{dw}^{\text{corrected}} + \epsilon}} \quad b := b - \alpha \frac{v_{db}^{\text{corrected}}}{\sqrt{s_{db}^{\text{corrected}} + \epsilon}}$$

- Hyperparameter choice:
 - α needs to be tuned, β_1 0.9 (dW/db), β_2 0.999 (dW^2/db^2), $\epsilon = 10^{-8}$
 - β_1 and β_2 can be tuned, but often aren't
 - β_1 computes mean of derivatives (first moment), β_2 calculated the exp weighted average of the squares (second moment) [hence moment estimation]

Learning Rate Decay

- Example use case:
 - Not converging because of noise in mini-batches
 - Larger steps at first to get towards the minimum, but then smaller steps to stay in the right region



- $\alpha = 1/(1 + \text{decayRate} * \text{epochNumber}) * \alpha_0$
- decayRate and α_0 are more hyperparameters to tune

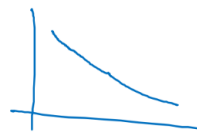
1 epoch = 1 pass through data.

$$\alpha = \frac{1}{1 + \text{decay-rate} * \text{epoch-num}} \alpha_0$$

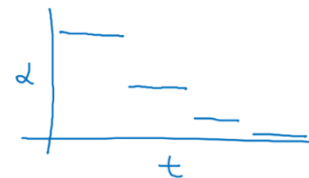
Epoch	α
1	0.1
2	0.67
3	0.5
4	0.4
⋮	⋮



$\alpha_0 = 0.2$
decay-rate = 1



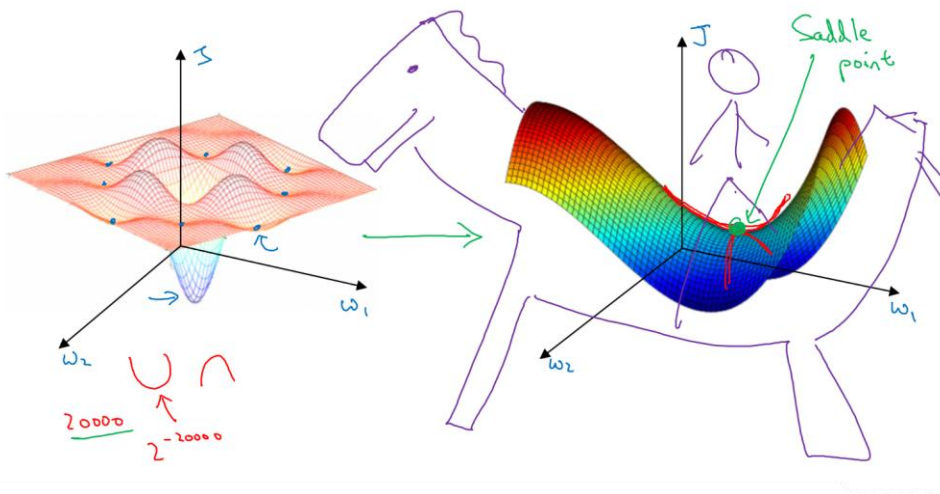
- Other learning rate methods:
 - Exponential decay: $\alpha = 0.95^{\text{epochNumber}} * \alpha_0$
 - $\alpha = k/\sqrt{\text{epochNumber}} * \alpha_0$ or $k/\sqrt{t} * \alpha_0$
 - Discrete staircase $\rightarrow$
 - Manual decay



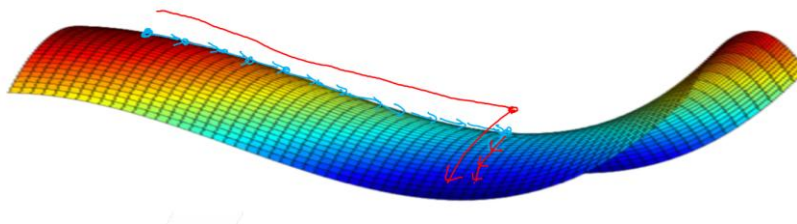
- Learning rate decay is lower in the list of things to try than have a fixed well-tuned α

The Problem of Local Optima

- When there are lots of local optima, it looks like a lot of places gradient descent could get stuck. However, when training a neural network, most of the points of zero gradients are not local optima; they are saddle points
- With a function of very high dimensional space, if the gradient is zero, then for it to be a local optimum, then all gradients from the point need to be the same direction (convex or concave) – unlikely. More likely some will go up and some will go down: saddle point
- Intuitions about low-dimensional space do not transfer to high dimensional spaces that our learning algorithms operate over
- Unlikely to get stuck in a bad local optimum



- If local optima aren't a problem, what is?
- **Plateaus** makes learning slow (derivative is close to zero for a long time)
 - Moments and RMSprop help by speeding up the rate at which you get down the plateau and off it



Which notation would you use to denote the 4th layer's activations when the input is the 7th example from the 3rd mini-batch?

$$a^{[3]\{7\}(4)}$$

$$a^{[4]\{3\}(7)}$$

$$a^{[7]\{3\}(4)}$$

Which of these statements about mini-batch gradient descent do you agree with?

One iteration of mini-batch gradient descent (computing on a single mini-batch) is faster than one iteration of batch gradient descent.

Training one epoch (one pass through the training set) using mini-batch gradient descent is faster than training one epoch using batch gradient descent. (same time)

You should implement mini-batch gradient descent without an explicit for-loop over different mini-batches, so that the algorithm processes all mini-batches at the same time (vectorization).

Suppose you don't face any memory-related problems. Which of the following make more use of vectorization?

Mini-Batch Gradient Descent with mini-batch size $m/2$.

Stochastic Gradient Descent, Batch Gradient Descent, and Mini-Batch Gradient Descent all make equal use of vectorization.

Batch Gradient Descent

Stochastic Gradient Descent

Why is the best mini-batch size usually not 1 and not m , but instead something in-between? Choose all that are true.

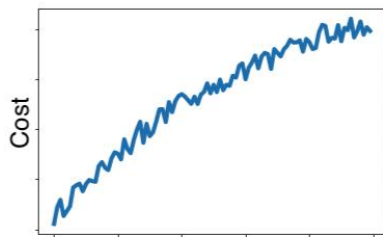
If the mini-batch size is 1, you end up having to process the entire training set before making any progress.

If the mini-batch size is 1, you lose the benefits of vectorization across examples in the mini-batch.

If the mini-batch size is m , you end up with batch gradient descent, which has to process the whole training set before making progress.

If the mini-batch size is m , you end up with stochastic gradient descent, which is usually slower than mini-batch gradient descent.

While using mini-batch gradient descent with a batch size larger than 1 but less than m the plot of the cost function J looks like this:



Which of the following do you agree with?

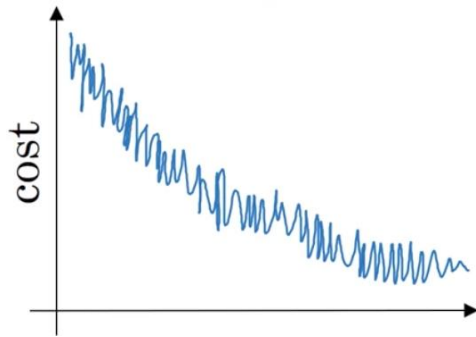
If you are using mini-batch gradient descent, this looks acceptable. But if you're using batch gradient descent, something is wrong.

If you are using batch gradient descent, this looks acceptable. But if you're using mini-batch gradient descent, something is wrong.

If you are using mini-batch gradient descent or batch gradient descent this looks acceptable.

No matter if using mini-batch gradient descent or batch gradient descent something is wrong. (squiggling is fine for mini-batch, but should be trending down not up)

Suppose your learning algorithm's cost J , plotted as a function of the number of iterations, looks like this:



Which of the following do you agree with?

Whether you're using batch gradient descent or mini-batch gradient descent, this looks acceptable.

Whether you're using batch gradient descent or mini-batch gradient descent, something is wrong.

If you're using mini-batch gradient descent, this looks acceptable. But if you're using batch gradient descent, something is wrong.

If you're using mini-batch gradient descent, something is wrong. But if you're using batch gradient descent, this looks acceptable.

Suppose the temperature in Casablanca over the first two days of March are the following:

March 1st: $\theta_1 = 30^\circ \text{C}$ March 2nd: $\theta_2 = 15^\circ \text{C}$

Say you use an exponentially weighted average with $\beta = 0.5$ to track the temperature: $v_0 = 0$, $v_t = \beta v_{t-1} + (1 - \beta) \theta_t$. If v_2 is the value computed after day 2 without bias correction, and $v_2^{\text{corrected}}$ is the value you compute with bias correction, what are these values?

$$v_2 = 20, v_2^{\text{corrected}} = 15 \quad \mathbf{v_2 = 15, v_2^{\text{corrected}} = 20}$$

$$v_2 = 20, v_2^{\text{corrected}} = 20 \quad v_2 = 15, v_2^{\text{corrected}} = 15$$

$$v_1 = \beta v_0 + (1 - \beta) \theta_1 = 0.5(0) + 0.5(30) = 15$$

$$v_2 = \beta v_1 + (1 - \beta) \theta_2 = 0.5(15) + 0.5(15) = 7.5 + 7.5 = \mathbf{15}$$

$$\text{Bias correction: } v_2^{\text{corrected}} = v_2 / (1 - \beta^t) = 15 / (1 - 0.5^2) = \mathbf{20}$$

Which of these is NOT a good learning rate decay scheme? Here, t is the epoch number.

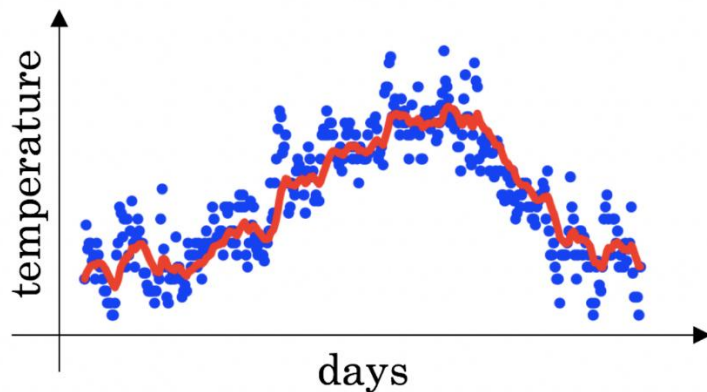
$$\alpha = \frac{1}{1 + 2 * t} \alpha_0$$

$$\alpha = e^t \alpha_0 \text{ (this doesn't decay)}$$

$$\alpha = 0.95^t \alpha_0$$

$$\alpha = \frac{1}{\sqrt{t}} \alpha_0$$

You use an exponentially weighted average on the London temperature dataset. You use the following to track the temperature: $v_t = \beta v_{t-1} + (1 - \beta) \theta_t$. The red line below was computed using $\beta = 0.9$. What would happen to your red curve as you vary β ? (Choose the two that apply).



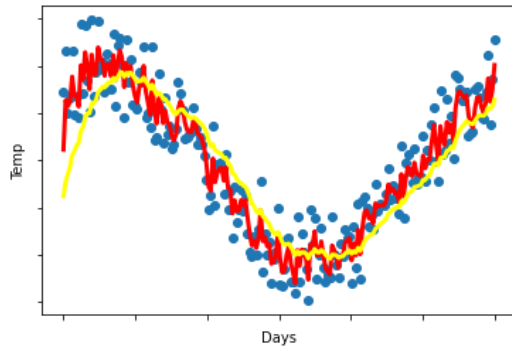
Decreasing β will shift the red line slightly to the right.

Increasing β will shift the red line slightly to the right.

Decreasing β will create more oscillation within the red line.

Increasing β will create more oscillations within the red line.

You use an exponentially weighted average on the London temperature dataset. You use the following to track the temperature: $v_t = \beta v_{t-1} + (1 - \beta)\theta_t$. The yellow and red lines were computed using values β_1 and β_2 respectively. Which of the following are true?



$\beta_1 = 0, \beta_2 > 0.$

$\beta_1 = \beta_2.$

$\beta_1 > \beta_2.$

$\beta_1 < \beta_2.$

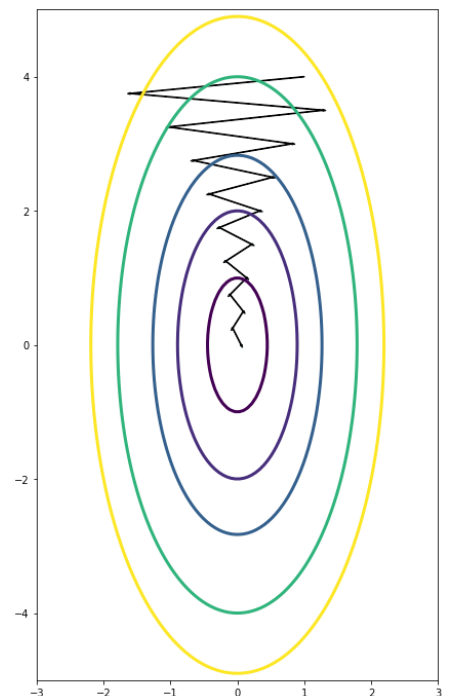
Consider the figure. Suppose this plot was generated with gradient descent with momentum $\beta = 0.01$. What happens if we increase the value of β to 0.10?

The gradient descent process starts oscillating in the vertical direction.

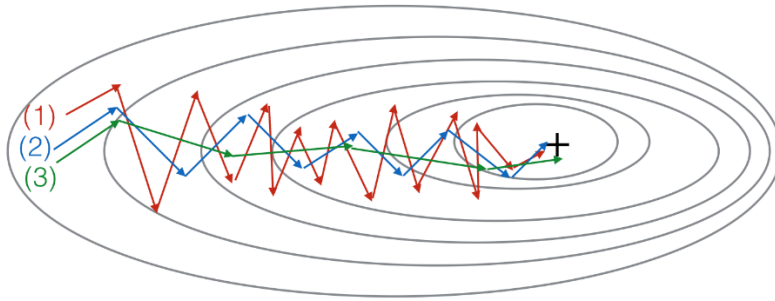
The gradient descent process starts moving more in the horizontal direction and less in the vertical.

The gradient descent process moves less in the horizontal direction and more in the vertical direction.

The gradient descent process moves more in the horizontal and the vertical axis.



Consider this figure:



These plots were generated with gradient descent; with gradient descent with momentum ($\beta = 0.5$); and gradient descent with momentum ($\beta = 0.9$). Which curve corresponds to which algorithm?

(1) is gradient descent. (2) is gradient descent with momentum (small β). (3) is gradient descent with momentum (large β)

(1) is gradient descent with momentum (small β). (2) is gradient descent. (3) is gradient descent with momentum (large β)

(1) is gradient descent with momentum (small β), (2) is gradient descent with momentum (small β), (3) is gradient descent

(1) is gradient descent. (2) is gradient descent with momentum (large β) . (3) is gradient descent with momentum (small β)

Suppose batch gradient descent in a deep network is taking excessively long to find a value of the parameters that achieves a small value for the cost function $J(W^{[1]}, b^{[1]}, \dots, W^{[L]}, b^{[L]})$. Which of the following techniques could help find parameter values that attain a small value for J ? (Choose all that apply) .

Normalize the input data.

Try using Adam.

Try initializing the weight at zero.

Try mini-batch gradient descent.

Which of the following are true about Adam?

Adam combines the advantages of RMSProp and momentum.

Adam can only be used with batch gradient descent and not with mini-batch gradient descent.

Adam automatically tunes the hyperparameter α alpha.

The most important hyperparameter on Adam is ϵ epsilon and should be carefully tuned.

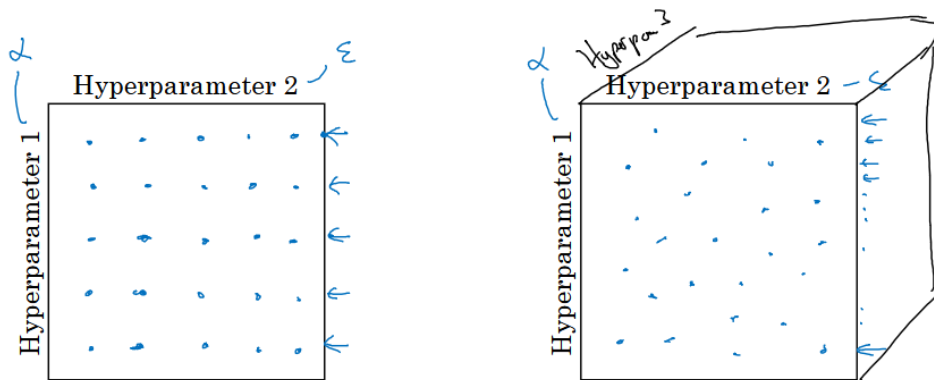
In very high dimensional spaces, it is most likely that the gradient descent process gives us a local minimum than a saddle point of the cost function. True/False?

True

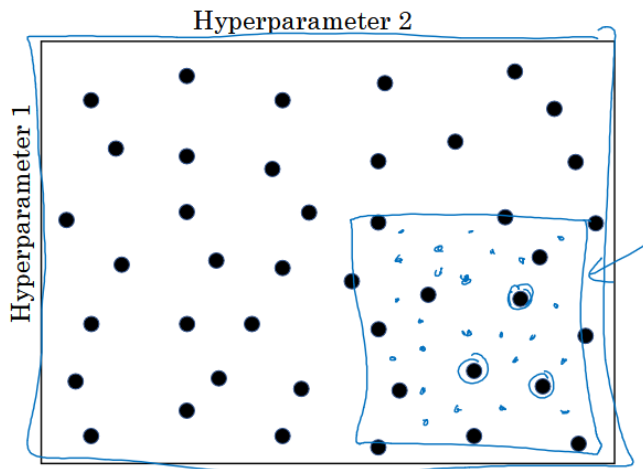
False

Hyperparameter Tuning Process

- Hyperparameters: α , β (momentum), β_1 , β_2 , ϵ (Adam), # layers, # **hidden units**, learning rate decay, **mini-batch size**
- Try random values: don't use a grid to systematically explore if you have a lot of hyperparameters. Choose the points at random. Some choices of hyperparameter matter more than others (α vs ϵ)



- Coarse to fine sampling
 - Zoom into a smaller region where those values worked. Focus more resources in searching the subspace

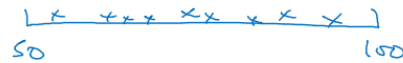


- Random sampling, coarse to fine search

Appropriate Scale

- Sampling at random doesn't mean sampling uniformly over the range of values
- Picking hyperparameters at random
 - Doesn't work for all hyperparameters

$$\rightarrow n^{test} = 50, \dots, 100$$

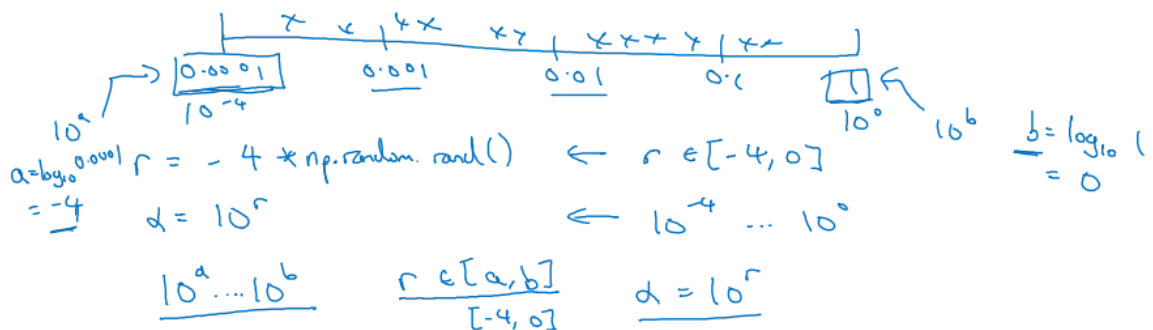
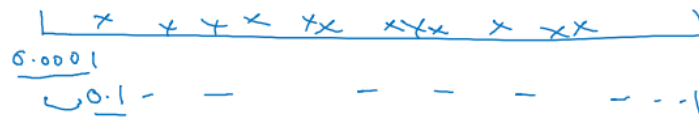


$$\rightarrow \# \text{ layers } L: 2 - 4$$

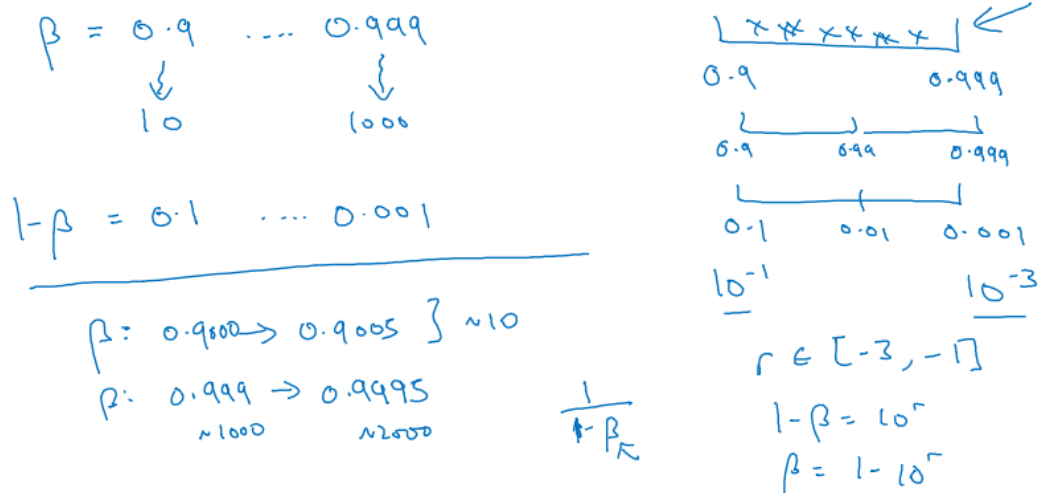
$$2, 3, 4$$

- Appropriate scale for hyperparameters
 - Uniform random search for α would only allow 10% of resources to search between 0.001 and 0.1. Better to search uniform on a log scale
 - Python: $r = -4 * \text{np.random.rand}(1) \leftarrow r$ random number between -4 and 0
 $\alpha = 10^r \leftarrow$ random number between 10^{-4} and 10^0
 - Sampling between 10^a and 10^b on log scale: determine a and b by taking the $\log_{10}$ of upper and lower α , sample r uniformly at random between a and b, then set α as 10^r

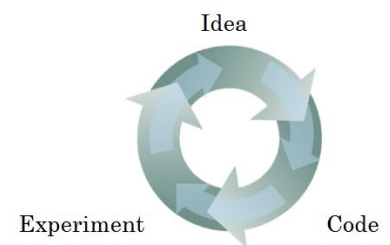
$$\alpha = 0.0001, \dots, 1$$



- Hyperparameters for exponentially weighted averages
 - Using $\beta=0.9$ is like averaging over last 10 values, 0.999 is like averaging over 1000
 - Searching on a linear scale doesn't make sense
 - Search over $1-\beta$, which ranges from 0.1 to 0.001
 - $a = 10^{-1}$, $b = 10^{-3}$, sample r uniformly at random from -3 to -1, then set $1-\beta = 1-10^r$
 - When β is close to 1, the sensitivity of results changes more rapidly (exp)
 - 0.999 avg over 1000 values, 0.9995 avg over 2000 values

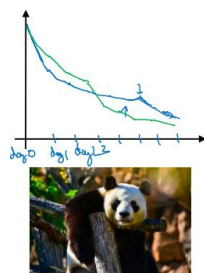


- Re-test hyperparameters occasionally
 - NLP, vision (Confonets or ResNets), speech, ads, logistics, ...
 - Intuitions get stale. Re-evaluate occasionally



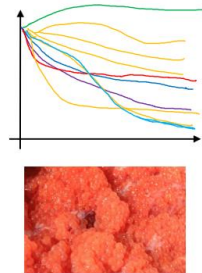
- Two major schools of thought
 - Babysitting one model: big dataset but not a lot of computational resources, nudge parameters over time (analogy: mammal offspring care)
 - Training many models in parallel: generate learning curves from different sets of hyperparameters (analogy: fish offspring bulk)

Babysitting one model



Panda

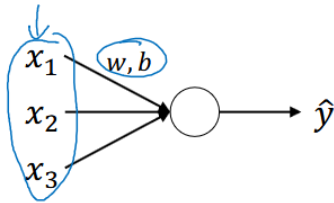
Training many models in parallel



Caviar

Normalizing Activations in a Network

- Normalizing inputs to speed up learning
 - Contours of learning problem elongated $\rightarrow$ round
 - Normalize mean and variance of activations (in practice, $z^{[l]}$) (as inputs to the next layer)



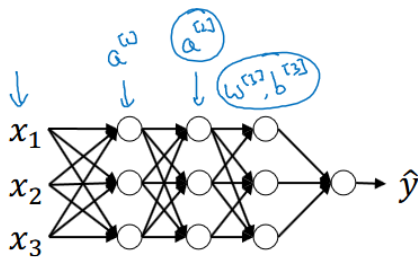
$$\mu = \frac{1}{m} \sum_i x^{(i)}$$

$$X = X - \mu$$

$$\sigma^2 = \frac{1}{m} \sum_i x^{(i)2}$$

$$X = X / \sigma$$

element-wise



Can we normalize $\frac{a^{[2]}}{w^{[2]}, b^{[2]}}$ so as to train faster?

Normalize $\frac{z^{[2]}}{\uparrow}$

- Implementing batch norm
 - Given some intermediate values in neural network and hidden unit values $z^{[l](1)}, \dots, z^{[l](m)}$, compute the mean as $\mu = 1/m \sum z^{[l](i)}$ and variance as $\sigma^2 = 1/m \sum (z^{[l](i)} - \mu)^2$, then normalize each $z^{[l](i)}$ as $z^{[l](i)}_{\text{norm}} = (z^{[l](i)} - \mu) / \sqrt{\sigma^2 + \epsilon}$ [for numerical stability]
 - We don't want the hidden units to always have mean 0 and variance 1, so compute $\tilde{z}^{[l](i)} = \gamma z^{[l](i)}_{\text{norm}} + \beta$ [γ and β are learnable parameters of model, which get updated alongside W and b . They allow you to set the mean and variance to whatever you want it to be. If $\gamma = \sqrt{\sigma^2 + \epsilon}$ and $\beta = \mu$, inverts normalization].

Given some intermediate values in NN $z^{(1)}, \dots, z^{(m)}$

$$\mu = \frac{1}{m} \sum_i z^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_i (z^{(i)} - \mu)^2$$

$$z^{(i)}_{\text{norm}} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\tilde{z}^{(i)} = \gamma z^{(i)}_{\text{norm}} + \beta$$

learnable parameters of model.

If $\gamma = \sqrt{\sigma^2 + \epsilon}$

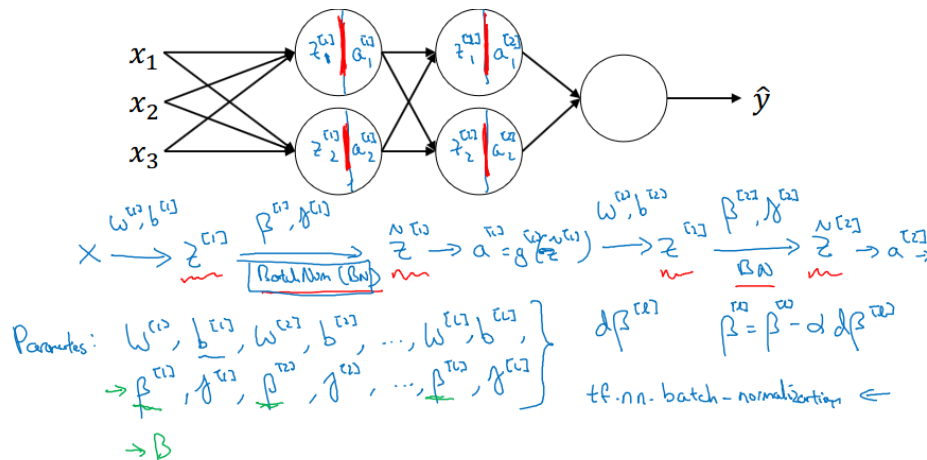
$\beta = \mu$

then $\tilde{z}^{(i)} = z^{(i)}$

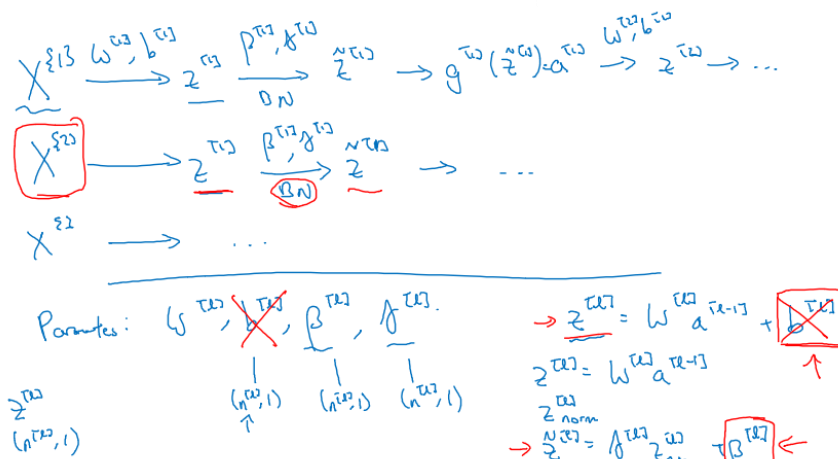
Use $\tilde{z}^{[l](i)}$ instad of $z^{[l](i)}$

Fitting Batch Norm into a Neural Network

- Each hidden unit computes two things: $z^{[l]}$ and $a^{[l]}$. Batch norm happens between
- $x \rightarrow (W^{[1]}, b^{[1]}) \rightarrow z^{[1]} \rightarrow \text{Batch Norm } (\beta^{[1]}, \gamma^{[1]}) \rightarrow z^{\sim[1]} \rightarrow a^{[1]} = g^{[1]}(z^{\sim[1]}) \rightarrow (W^{[2]}, b^{[2]}) \rightarrow z^{[2]} \rightarrow \dots$
- Parameters: $W^{[1]}, b^{[1]}, \dots, W^{[L]}, b^{[L]}, \beta^{[1]}, \gamma^{[1]}, \dots, \beta^{[L]}, \gamma^{[L]}$
- Different β than momentum, RMSprop, and Adam algorithms
- Use whatever optimization you want (gradient descent) to implement the algorithm
 - Ex: Compute $d\beta^{[l]}$ and update the parameter as $\beta^{[l]} = \beta - \alpha d\beta^{[l]}$
- Usually don't have to implement batch norm step yourself, frameworks have options
 - Ex: TensorFlow: `tf.nn.batch-normalization`



- Working with mini-batches
 - $X^{[1]} \rightarrow (W^{[1]}, b^{[1]}) \rightarrow z^{[1]} \rightarrow \text{BN } (\beta^{[1]}, \gamma^{[1]}) \rightarrow z^{\sim[1]} \rightarrow a^{[1]} = g^{[1]}(z^{\sim[1]}) \rightarrow (W^{[2]}, b^{[2]}) \rightarrow z^{[2]} \dots$
 - $X^{[2]} \rightarrow (W^{[1]}, b^{[1]}) \rightarrow z^{[1]} \rightarrow \text{BN } (\beta^{[1]}, \gamma^{[1]}) \rightarrow z^{\sim[1]} \rightarrow a^{[1]} = g^{[1]}(z^{\sim[1]}) \rightarrow (W^{[2]}, b^{[2]}) \rightarrow z^{[2]} \dots$
 - Parameters: $W^{[l]}, b^{[l]}, \beta^{[l]}, \gamma^{[l]}$
 - $z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$ ($b^{[l]}$ subtracted out by rescale) $\rightarrow z^{\sim[l]} = \gamma^{[l]} z_{\text{norm}}^{[l]} + \beta^{[l]}$ [use $\beta^{[l]}$ to determine the mean of $z^{[l]}$, which is what gets passed to the next layer]

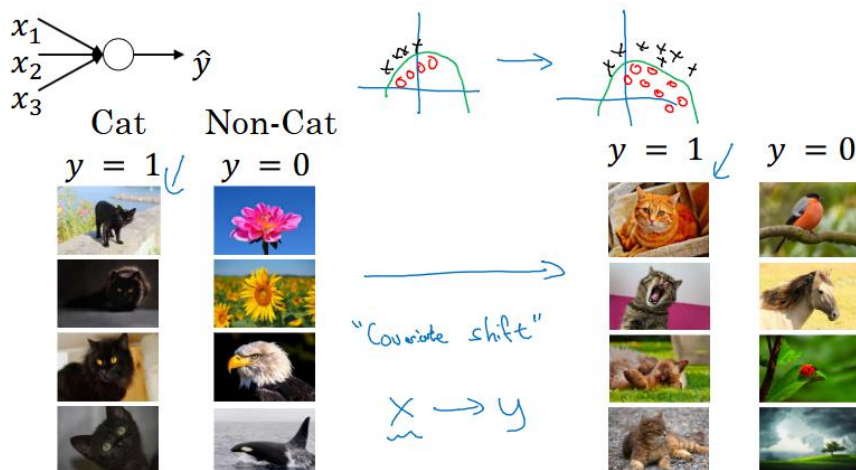


- Implementing gradient descent
 - For each minibatch: forward prop on $X^{(t)}$, in each hidden layer use batch norm to replace $z^{[l]}$ with $\tilde{z}^{[l]}$, use backprop to compute derivatives, update parameters
 - Works with momentum, RMSprop, Adam

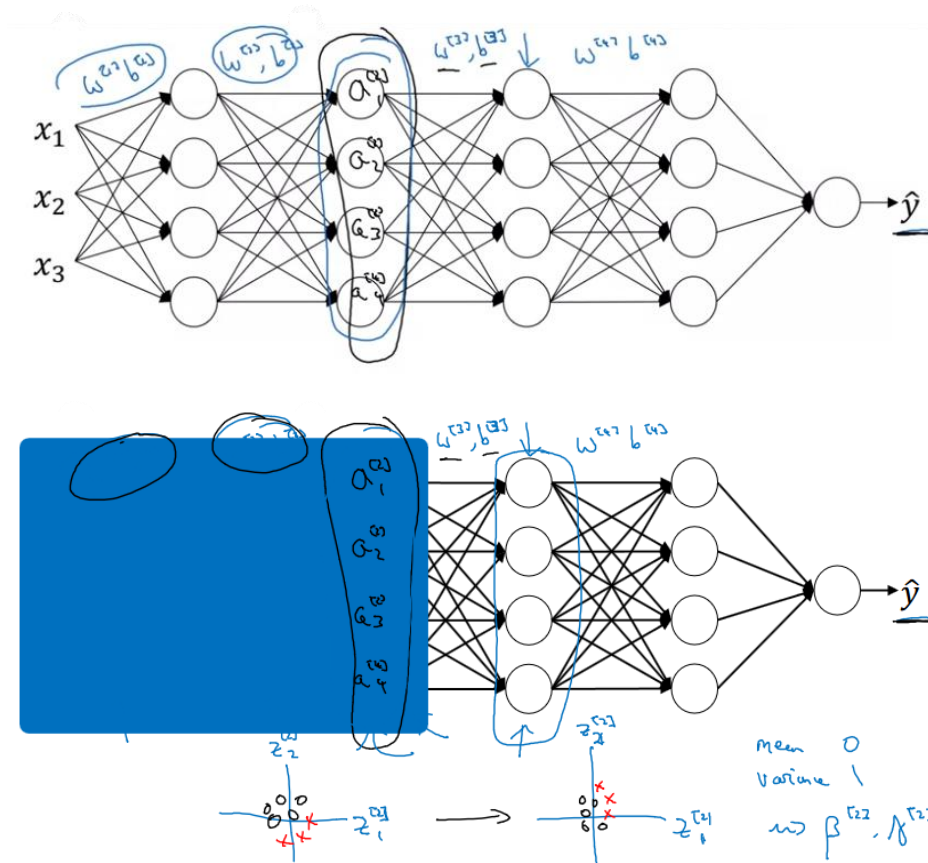
for $t = 1 \dots \text{num Minibatches}$
 Compute forward prop on $X^{(t)}$.
 In each hidden layer, use BN to repa $z^{(t)}$ with $\tilde{z}^{(t)}$.
 Use backprop to compute $\frac{dL}{dW^{(t)}}$, ~~$\frac{dL}{d\alpha^{(t)}}$~~ , $\frac{dL}{d\beta^{(t)}}$, $\frac{dL}{df^{(t)}}$
 Update pointers $W^{(t)} := W^{(t)} - \alpha \frac{dL}{dW^{(t)}}$
 $\beta^{(t)} := \beta^{(t)} - \alpha \frac{dL}{d\beta^{(t)}}$
 $f^{(t)} := \dots$ } $\leftarrow$
 Works w/ momentum, RMSprop, Adam.

Why Batch Norm Works

- Makes weights deeper in NN more robust to changes in earlier layers
- Learning on shifting input distribution (**covariate shift**)
- May need to retrain the network, even if the function is the same (ex 1 or 0)
 - Ex: trained on black cats, evaluated on colored cats



- Why is this a problem with neural networks?
 - From the perspective of the third hidden layer, the hidden unit values are changing all the time. It's suffering from the problem of covariate shift
 - Batch norm reduces the amount that the distribution of hidden unit values shifts around. No matter how it changes, the mean and variance will remain the same
 - Weakens the coupling between early layer and later layer parameters $\rightarrow$ each layer learns a little more independently of other layers in NN $\rightarrow$ faster learning



- Batch Norm as regularization (side effect)
 - Each mini-batch is scaled by the mean/variance computed on just that mini-batch $\rightarrow$ mean/variance has a little bit of noise in it
 - This adds some noise to the values $z^{(l)}$ within that minibatch. So similar to dropout, it adds some noise to each hidden layer's activations
 - This has a slight regularization effect
 - Bigger mini-batch size $\rightarrow$ reducing noise $\rightarrow$ reducing regularization effect
 - BN not intended to be used as regularization method

- Batch Norm at test time

- BN process data one mini-batch at a time, but at test time you might need to process the examples one at a time
- μ and σ^2 are calculated on the entire mini-batch. Don't want this for testing (mean and variance of one example doesn't make sense). Estimate using exp weighted average across mini-batches
- Robust to exact method of estimating μ and σ^2 (frameworks typically have a default way)

$$\mu = \frac{1}{m} \sum_i z^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_i (z^{(i)} - \mu)^2$$

$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\tilde{z}^{(i)} = \gamma z_{\text{norm}}^{(i)} + \beta$$

μ, σ^2 : estimate using exponentially weighted average (across mini-batches).

$x^{(1)}, x^{(2)}, x^{(3)}, \dots$

$\downarrow$

$\mu^{(1)}, \mu^{(2)}, \mu^{(3)}, \dots \rightarrow \mu$

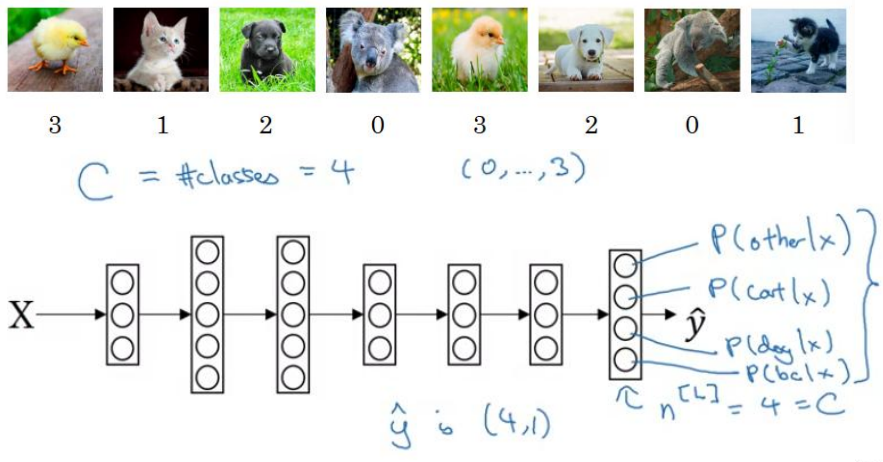
$\sigma^{(1)}, \sigma^{(2)}, \sigma^{(3)}, \dots \rightarrow \sigma^2$

$\tilde{z}_{\text{norm}} = \frac{z - \mu}{\sqrt{\sigma^2 + \epsilon}}$

$\tilde{z} = \gamma \tilde{z}_{\text{norm}} + \beta$

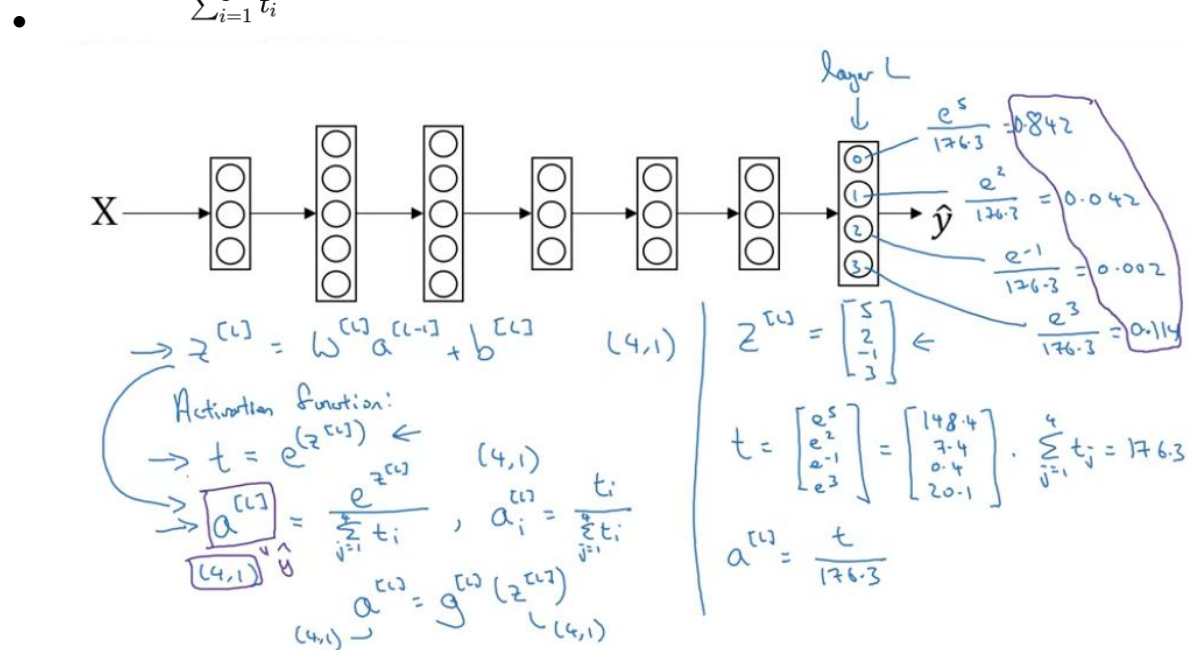
Softmax Regression

- Softmax regression is a generalization of logistic regression for recognizing one of multiple (>2) classes. With $C=2$ classes, softmax reduces to logistic
- Multi-class classification
- Output layer has C ($C = \# \text{ classes}$) output units. $\hat{y}$ is $(4, 1)$. The elements of $\hat{y}$ (probabilities) should sum to 1



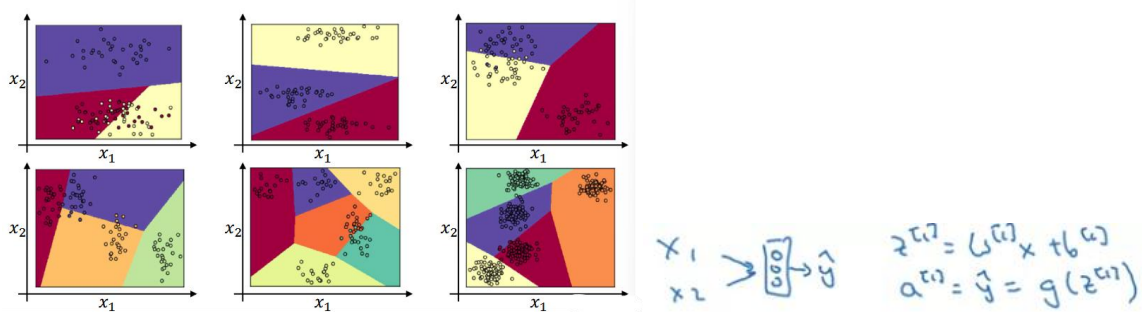
- Compute a temporary variable $t = e^{z^{[L]}}$ and output $a^{[L]} = e^{z^{[L]}} / \sum_i t_i$ [t normalized to sum to 1].
 $a_i^{[L]} = t_i / \sum t_i$

$$a^{[L]} = \frac{e^{z^{[L]}}}{\sum_{i=1}^4 t_i}$$



- Takes a $(C, 1)$ vector and outputs a $(C, 1)$ vector (ReLU real # $\rightarrow$ real #)

- Examples:



- Decision boundaries are linear

Training a Softmax Classifier

- Understanding softmax
 - Biggest element of $z^{(L)}$ maps to the largest probability value in $a^{(L)}$

Handwritten example for $C=4$:

$$z^{(L)} = \begin{bmatrix} 5 \\ 2 \\ -1 \\ 3 \end{bmatrix} \quad t = \begin{bmatrix} e^5 \\ e^2 \\ e^{-1} \\ e^3 \end{bmatrix}$$

Soft max:

$$a^{(L)} = g^{(L)}(z^{(L)}) = \begin{bmatrix} e^5 / (e^5 + e^2 + e^{-1} + e^3) \\ e^2 / (e^5 + e^2 + e^{-1} + e^3) \\ e^{-1} / (e^5 + e^2 + e^{-1} + e^3) \\ e^3 / (e^5 + e^2 + e^{-1} + e^3) \end{bmatrix} = \begin{bmatrix} 0.842 \\ 0.042 \\ 0.002 \\ 0.114 \end{bmatrix}$$

Hard max:

$$\hat{y} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

- Softmax loss function
 - $L(\hat{y}, y) = -\sum y_j \log(\hat{y}_j)$ $J(W^{[1]}, b^{[1]}, \dots) = 1/m \sum L(\hat{y}^{(i)}, y^{(i)})$
 - Make probability of ground truth class in training set as high as possible (MLE)

Handwritten derivation of the softmax loss function:

For a single example i with $C=4$:

$$y^{(i)} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad \hat{y}^{(i)} = \begin{bmatrix} 0.3 \\ 0.2 \\ 0.1 \\ 0.4 \end{bmatrix}$$

Loss calculation:

$$L(\hat{y}, y) = -\sum_{j=1}^C y_j \log \hat{y}_j$$

Since $y_1 = y_3 = y_4 = 0$, the loss is:

$$-y_2 \log \hat{y}_2 = -\log \hat{y}_2$$

Goal: Make $\hat{y}_2$ big.

Overall loss function:

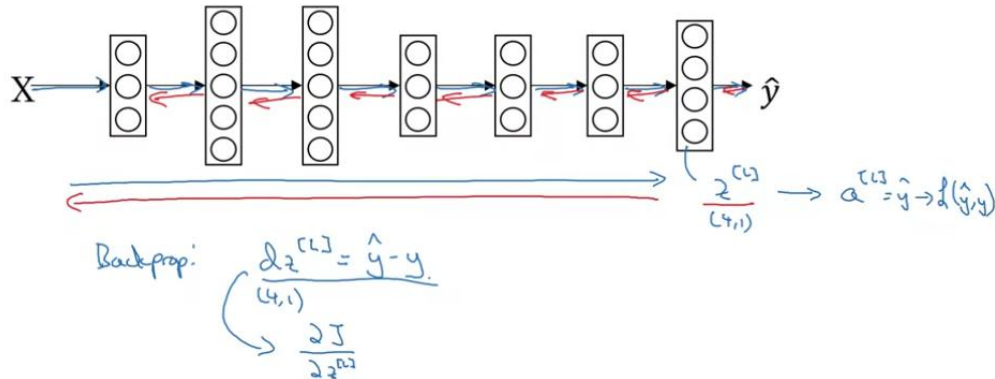
$$J(W^{(1)}, b^{(1)}, \dots) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

Matrix notation:

$$Y = \begin{bmatrix} y^{(1)} & y^{(2)} & \dots & y^{(m)} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 & \dots \\ 1 & 0 & 0 & 0 & \dots \\ 0 & 1 & 0 & 0 & \dots \\ 0 & 0 & 0 & 0 & \dots \end{bmatrix}$$

$$\hat{Y} = \begin{bmatrix} \hat{y}^{(1)} & \dots & \hat{y}^{(m)} \end{bmatrix} = \begin{bmatrix} 0.3 & \dots & 0.2 \\ 0.2 & \dots & 0.1 \\ 0.1 & \dots & 0.4 \\ 0.4 & \dots & \dots \end{bmatrix}$$

- Implementing gradient descent with softmax
 - $dz^{[L]} = \hat{y} - y = dJ/dz^{[L]}$
 - Programming framework often figures out how to do backprop for you given your specified forward prop



Deep Learning Frameworks

- Good to understand the mechanics, but use frameworks for efficient DL
- Caffe(2), CNTK, DL4J, Keras, Lasagne, mxnet, PaddlePaddle, TensorFlow, Theano, Torch
- Choosing deep learning frameworks: ease of programming (development and deployment), running speed, truly open (open source with good governance)

TensorFlow

- Motivating problem

$$J(w) = w^2 - 10w + 25 = (w-5)^2 \rightarrow w = 5 \text{ minimizes}$$

What if you have some complicated cost function $J(W, b)$ that doesn't factor nicely?

- GradientTape in Python records sequence of operations while computing cost function in forward prop, and can revisit in reverse order and compute backprop and gradients
- Python implementation:

```
import numpy as np
import tensorflow as tf

w = tf.Variable(0, dtype=tf.float32)
optimizer = tf.keras.optimizers.Adam(0.1)

def train_step():
    with tf.GradientTape() as tape:
        cost = w ** 2 - 10 * w + 25
        trainable_variables = [w]
        grads = tape.gradient(cost, trainable_variables)
        optimizer.apply_gradients(zip(grads, trainable_variables))

print(w)
Output: <tf.Variable 'Variable:0' shape=() dtype=float32, numpy=0.0>

train_step()
print(w)
Output: <tf.Variable 'Variable:0' shape=() dtype=float32,
numpy=0.099999997>

for i in range(1000)
    train_step()
print(w)
Output: <tf.Variable 'Variable:0' shape=() dtype=float32,
numpy=5.0000001>
```

We knew 5 was the min of this function!

- What is the function you want to minimize is also a function of the training set X?
 - Can implement backprop without specifying gradient tape

```
w = tf.Variable(0, dtype=tf.float32)
x=np.array([1.0, -10.0, 25.0], dtype=np.float32) #cost fxn coefficients
optimizer = tf.keras.optimizers.Adam(0.1)
```

```
def cost_fxn():
    return cost = x[0] * w ** 2 + x[1] * w + x[2]
print(w)
Output: <tf.Variable 'Variable:0' shape=() dtype=float32, numpy=0.0>
```

```
optimizer.minimize(cost_fxn, [w]) # single line substitute for above
print(w)
Output: <tf.Variable 'Variable:0' shape=() dtype=float32,
numpy=0.099999997>
```

```
for i in range(1000)
    train_step()
print(w)
Output: <tf.Variable 'Variable:0' shape=() dtype=float32,
numpy=5.0000001>
```

```
def training(x, w, optimizer):
    def cost_fxn():
        return cost = x[0] * w ** 2 + x[1] * w + x[2]
    for i in range(1000):
        optimizer.minimize(cost_fxn, [w])
    return w
```

```
w = training(x, w, optimizer)
print(w)
Output: <tf.Variable 'Variable:0' shape=() dtype=float32,
numpy=5.0000001>
```

```
import numpy as np
import tensorflow as tf

w = tf.Variable(0, dtype=tf.float32)
x = np.array([1.0, -10.0, 25.0], dtype=np.float32)
optimizer = tf.keras.optimizers.Adam(0.1)

def training(x, w, optimizer):
    def cost_fn():
        return x[0] * w ** 2 + x[1] * w + x[2]
    for i in range(1000):
        optimizer.minimize(cost_fn, [w])
    return w

w = training(x, w, optimizer)
```



Which of the following are true about hyperparameter search?

Choosing values in a grid for the hyperparameters is better when the number of hyperparameters to tune is high since it provides a more ordered way to search.

When sampling from a grid, the number of values for each hyperparameter is larger than when using random values.

Choosing random values for the hyperparameters is convenient since we might not know in advance which hyperparameters are more important for the problem at hand.

When using random values for the hyperparameters they must be always uniformly distributed.

If it is only possible to tune two parameters from the following due to limited computational resources. Which two would you choose?

α

β_1, β_2 in Adam.

ϵ in Adam.

The β parameter of the momentum in gradient descent.

Even if enough computational power is available for hyperparameter tuning, it is always better to babysit one model ("Panda" strategy), since this will result in a more custom model. True/False?

False

True

Knowing that the hyperparameter α should be in the range of 0.00001 and 1.0, which of the following is the recommended way to sample a value for α ?

`r = np.random.rand()`

`alpha = 10**r`

`r = -5*np.random.rand()`

`alpha = 10**r`

`r = -4*np.random.rand()`

`alpha = 10**r`

`r = np.random.rand()`

`alpha = 0.00001 + r*0.99999`

Finding new values for the hyperparameters, once we have found good ones for a model, should only be done if new hardware or computational power is acquired. True/False?

True

False

In batch normalization as presented in the videos, if you apply it on the l th layer of your neural network, what are you normalizing?

$b^{[l]}$

$W^{[l]}$

$a^{[l]}$

$z^{[l]}$

Which of the following is true about batch normalization?

The parameter ϵ in the batch normalization formula is used to accelerate the convergence of the model.

One intuition behind why batch normalization works is that it helps reduce the internal covariance.

There is a global value of γ and β that is used for all the hidden layers where batch normalization is used.

The parameters β and γ of batch normalization can't be trained using Adam or RMS prop.

Which of the following are true about batch normalization?

$\beta^{[l]}$ and $\gamma^{[l]}$ are hyperparameters that must be tuned by random sampling in a logarithmic scale. (parameters, not hyperparameters)

The parameters $\gamma^{[l]}$ and $\beta^{[l]}$ set the variance and mean of $\tilde{z}^{[l]}$.

When using batch normalization, we introduce two new parameters $\gamma^{[l]}$, $\beta^{[l]}$ that must be "learned" or trained.

$$z_{norm}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2}} \cdot (\sigma^2 + \epsilon)$$

After training a neural network with Batch Norm, at test time, to evaluate the neural network on a new example you should:

Use the most recent mini-batch's value of μ and σ^2 to perform the needed normalizations.

Skip the step where you normalize using μ and σ^2 since a single test example cannot be normalized.

Perform the needed normalizations, use μ and σ^2 , squared estimated using an exponentially weighted average across mini-batches seen during training.

If you implemented Batch Norm on mini-batches of (say) 256 examples, then to evaluate on one test example, duplicate that example 256 times so that you're working with a mini-batch the same size as during training.

If a project is open-source, it is a guarantee that it will remain open source in the long run and will never be modified to benefit only one company. True/False?

False

Trues